Project Cassandra

for <Catch22>

Final Presentation



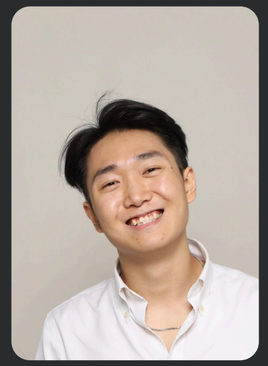
Presented by
G4 | TEAM PYGMY <3

32_IS483_PD_DCS_BA_Pygmy_Cassandra | Supervisor Nyan Tun Zaw

Arin | Isaiah | Jamesz | Jerrick | Joyce | Yan Ting

Meet Our Team

Leads



PRODUCT OWNER

Isaiah



SCRUM MASTER

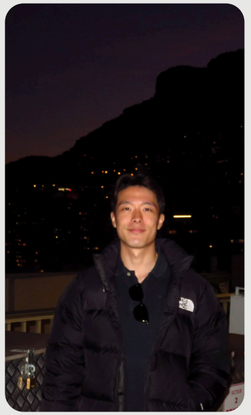
Jerrick

Developers



DEVELOPER

Arin



DEVELOPER

Jamesz



DEVELOPER

Yan Ting



DEVELOPER

Joyce

Today

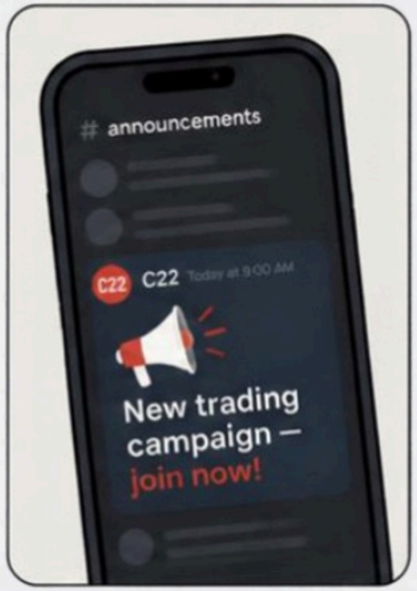
01	Recap
02	Goals & Impact
03	Demo
04	System Architecture
05	The Auto-Retrainer
06	Model Training
07	Explanations
08	Project Management
09	Handover

01

Recap

The problem, and what changed since midterms

THE WHOLE JOURNEY



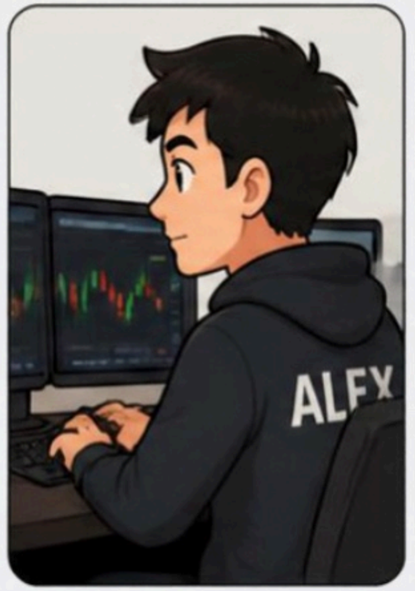
1. DISCORD ANNOUNCEMENT



2. SALLY JOINS



3. SALLY TRADES



4. ALEX MIRRORS IN THE REAL MARKET



5. THE JUDGE DECIDES

C22 can bet against Alex and hopefully profit.

The Judge decides when to.

THE CONSTRAINT

C22 Hit a Data Ceiling

500

Users per campaign

The cap while campaigns ran on TradeLocker

Two limits

What each campaign could teach the Judge

Size — Too few traders, so too few trades per campaign

Diversity — Too narrow a mix of trading styles to learn from

Clement, C22
10 Nov 2026

“Some of your findings line up with what I’ve been seeing independently… I think the model itself y’all work on is pretty good, is just maybe, like you said, the lack of data that can’t paint a better picture.”

THE CONSTRAINT

Limited Data Diversity

54.7%

of all trades

come from the top 10% of traders

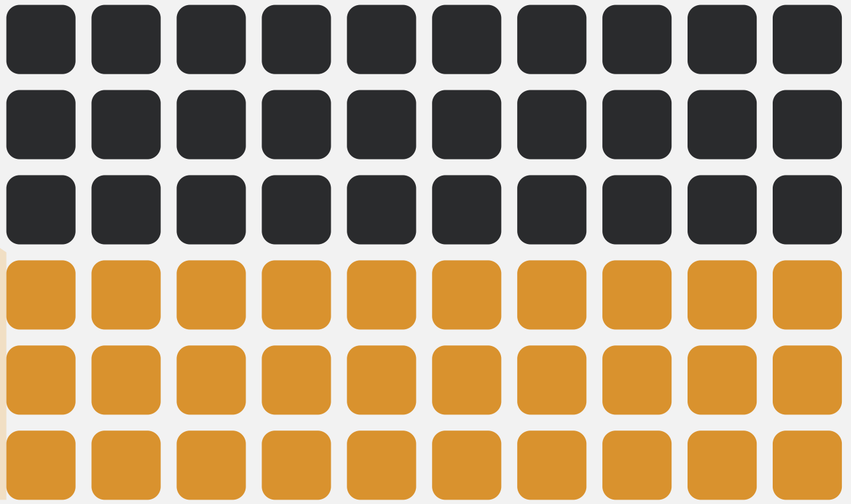


Traders in 10 groups, by trades placed

60 campaigns

of data

≈ 30 campaigns of variety



Everyone else

The same few traders again

WHAT IT MEANT

More Modelling Won’t Fix This

6/30

The Judge plateaued

Only 6 of the 30 variants we tried finished above $0. Swapping model type moved AUC by 0.002.

Profit = Direction × Size

Why magnitude didn’t help

Sizing only pays when direction is reliable, and ours is near chance. Two sizing rules moved profit by less than $130 a campaign.

The fix: **More data, not a bigger model.**

WHAT CHANGED

C22 Built Its Own Platform

More traders means more data

so the Judge can keep improving.

500

Users per campaign today

On TradeLocker

2,000

Users per campaign, target

On C22’s own platform. 4× as many

OUR POSITIONING

Make Retraining Easy

Our question:

“How do we retrain and promote the Judge as new data comes in?”

01

Retrain

A new batch of campaigns starts a training run automatically

02

Check

The new model is graded against the current one before anyone sees it

03

Promote

A person approves it. Every older model is kept for rollback

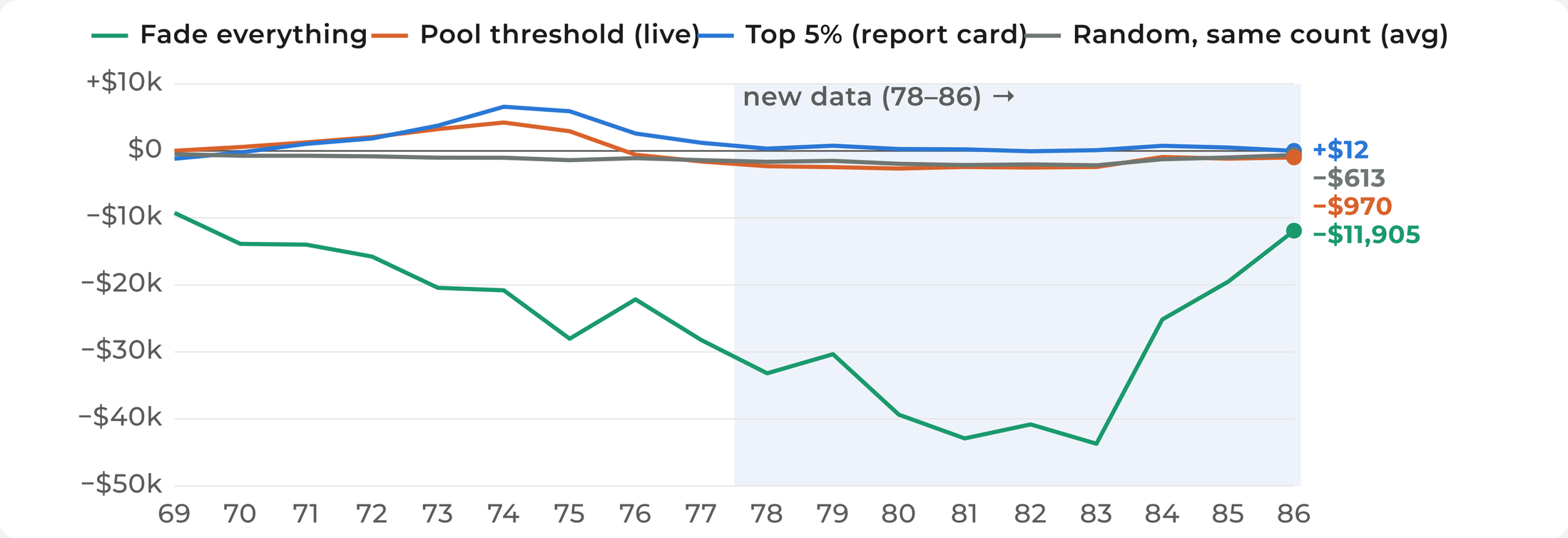
02

Goals & Impact

The requirements we committed to, and where we stand

GOALS & IMPACT

We beat 2 of the 3 benchmarks



Our top 5% ends at +$12; fade everything at −$11,905.
Results held since midterms, consistent with the data ceiling.

GOALS & IMPACT

What We Committed To

Benchmarks

From our midterm success metrics

- **Beat fade everything** — ✓ Beaten
- **Beat random** — ✓ Beaten
- **Match C22's rule** — Data limitations

Midterm promises

What we said we would build next

- **Magnitude** — Spiked, no performance impact
- **Paper trading** — ✓ Delivered
- **Auto-retrainer** — ✓ Delivered
- **Explanations** — ✓ Delivered

MID-TERM FEEDBACK

We Took the Feedback

Prof Kyong
Mid-term Q&A

“What has the model learned about traders that Clement doesn’t already know?”

ADDED

A reason for every trade

Two to four plain-English sentences per decision

HOW

Score attribution

TreeSHAP on the Judge’s dollar score, grouped into six families

NOT

Not an LLM

Fixed templates, so the same trade always gets the same explanation

ADDED VALUE

What C22 Gets Beyond the Judge

A system C22 can run without us

NO REAL MONEY

Paper trading

Every fade is recorded at the live gold price. Nothing goes to a broker.

ROOM TO GROW

Scale easily

Built for 500 → 2,000 users. Add workers with one command.

LESS RISK

Daily backups

Data and models, kept for a week. One command restores them.

LESS ADMIN

Easy imports

Import past campaigns from the dashboard. Bad rows are skipped and reported.

03

Demo

The system running end to end

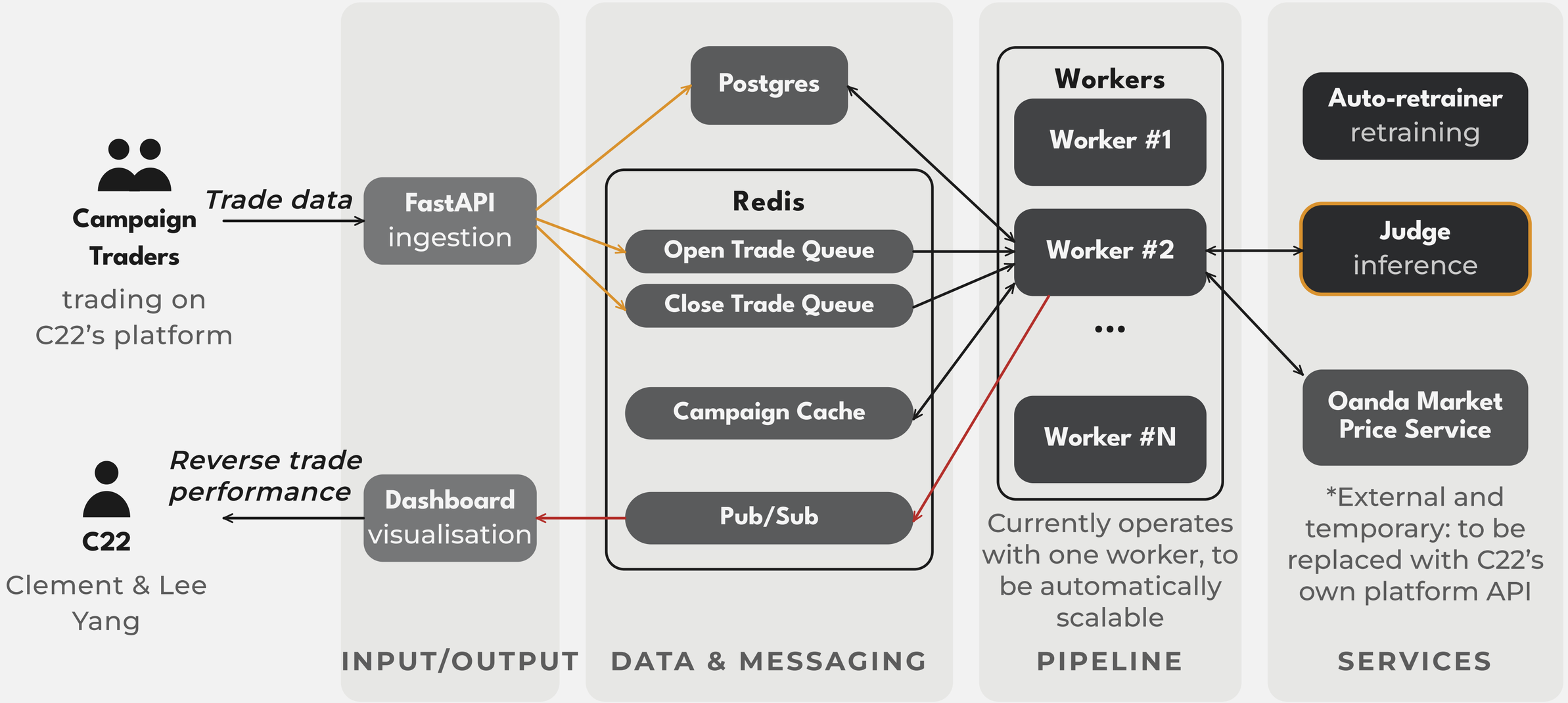
04

System Architecture

What we built, and where the retrainer fits

PROJECT CASSANDRA - DURING CAMPAIGN

Project Cassandra Architecture



AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1

8 unseen campaigns

A run starts

WHO DOES IT

Cassandra app

WHAT IT PRODUCES

Trigger to start retraining

AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1

8 unseen campaigns

A run starts

2

Prepare training data

Older 4 validate, newer 4 test

WHO DOES IT

Cassandra app

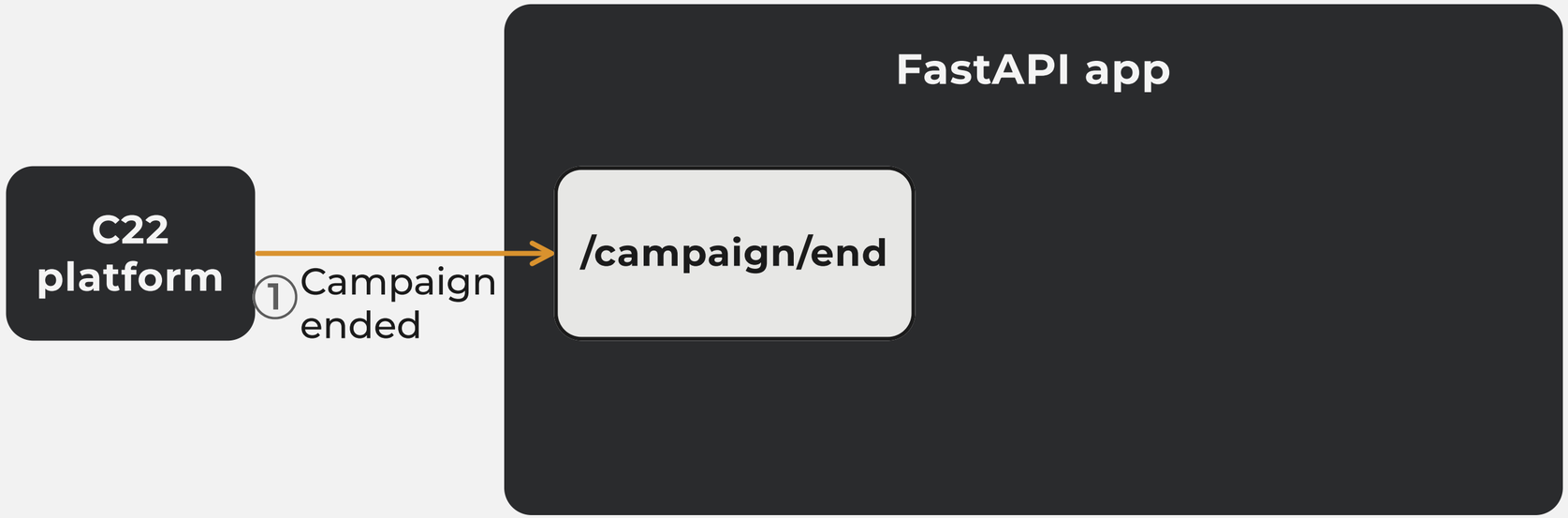
WHAT IT PRODUCES

Trigger to start retraining

Retraining dataset

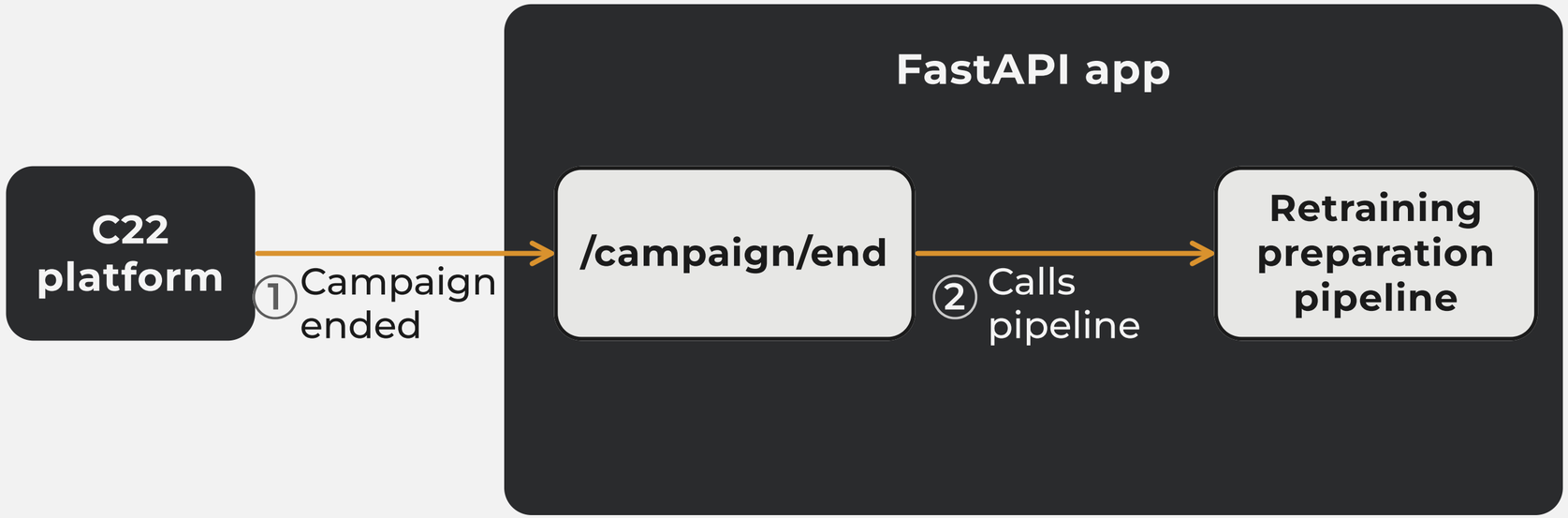
AUTO-RETRAINER

From Campaign End to Trainer



AUTO-RETRAINER

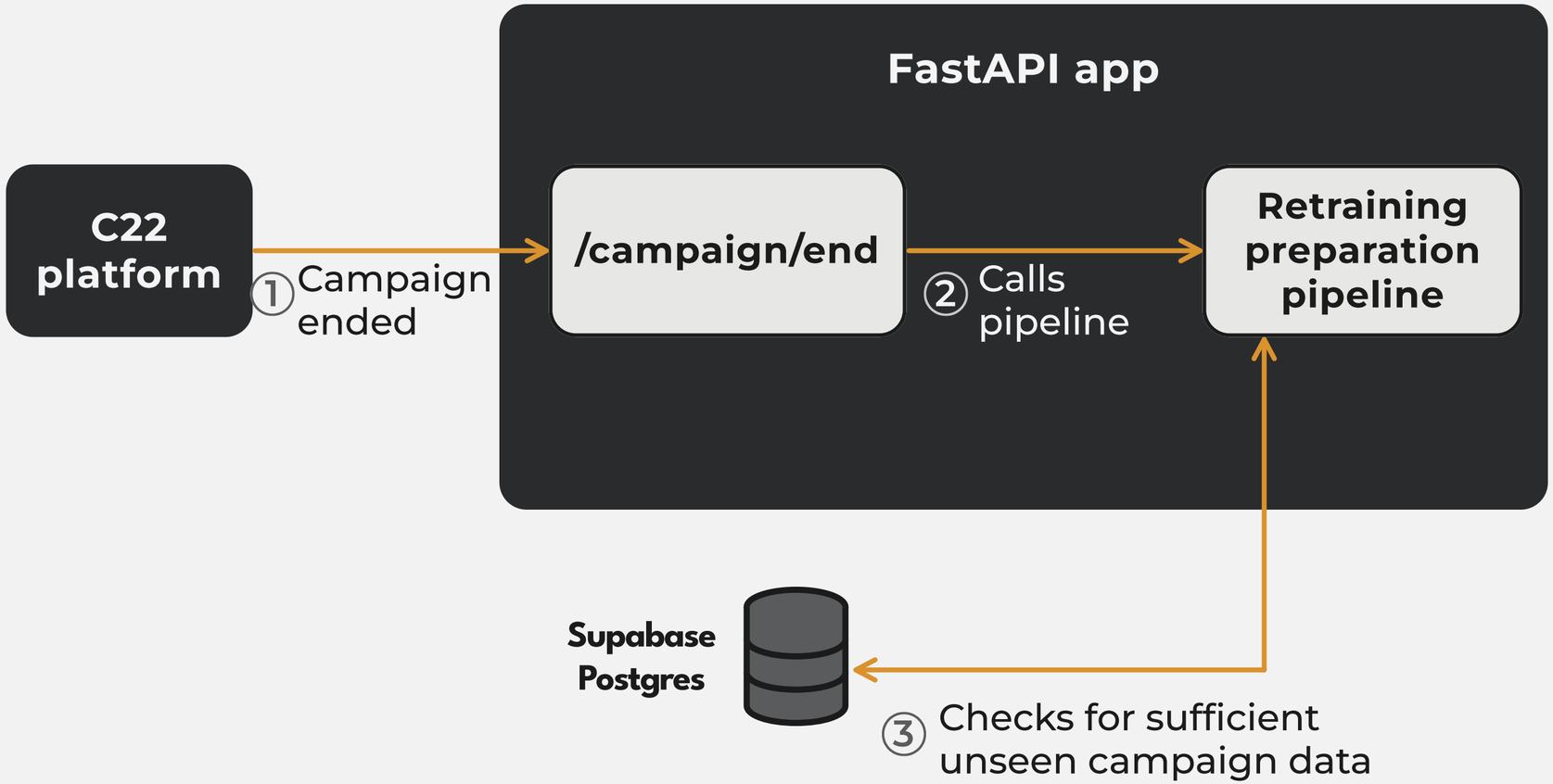
From Campaign End to Trainer



AUTO-RETRAINER

From Campaign End to Trainer

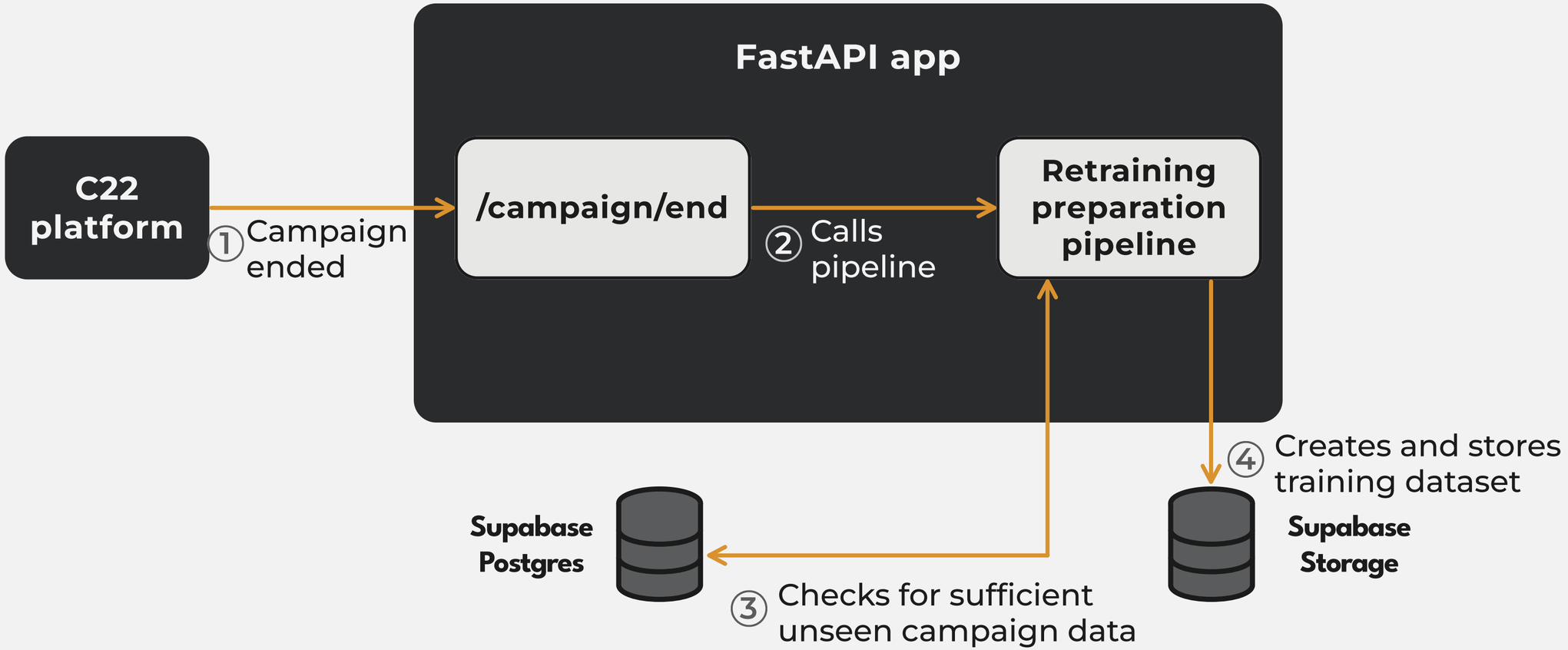
1 **8 unseen campaigns**
A run starts



AUTO-RETRAINER

From Campaign End to Trainer

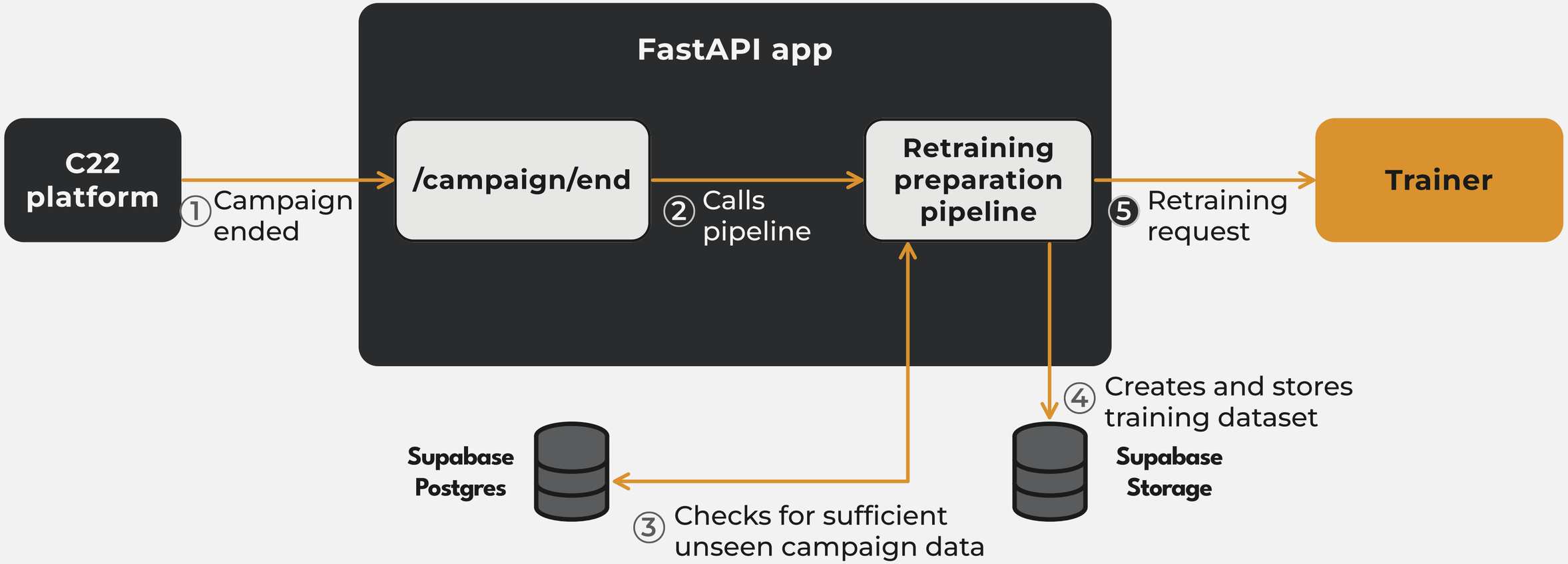
1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test



AUTO-RETRAINER

From Campaign End to Trainer

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test



AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1

8 unseen campaigns

A run starts

2

Prepare training data

Older 4 validate, newer 4 test

WHO DOES IT

Cassandra app

WHAT IT PRODUCES

Trigger to start retraining

Retraining dataset

AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test
3. **Train a challenger**
   In a separate container

WHO DOES IT

Cassandra app

Trainer

WHAT IT PRODUCES

Trigger to start retraining

Retraining dataset

A challenger model

AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test
3. **Train a challenger**
   In a separate container
4. **Grade it**
   Against the current champion

WHO DOES IT

- **Cassandra app**
- **Trainer**
- **Evaluator**

WHAT IT PRODUCES

- Trigger to start retraining
- Retraining dataset
- A challenger model
- Model promotion recommendation

AUTO-RETRAINER

From Trained to Live

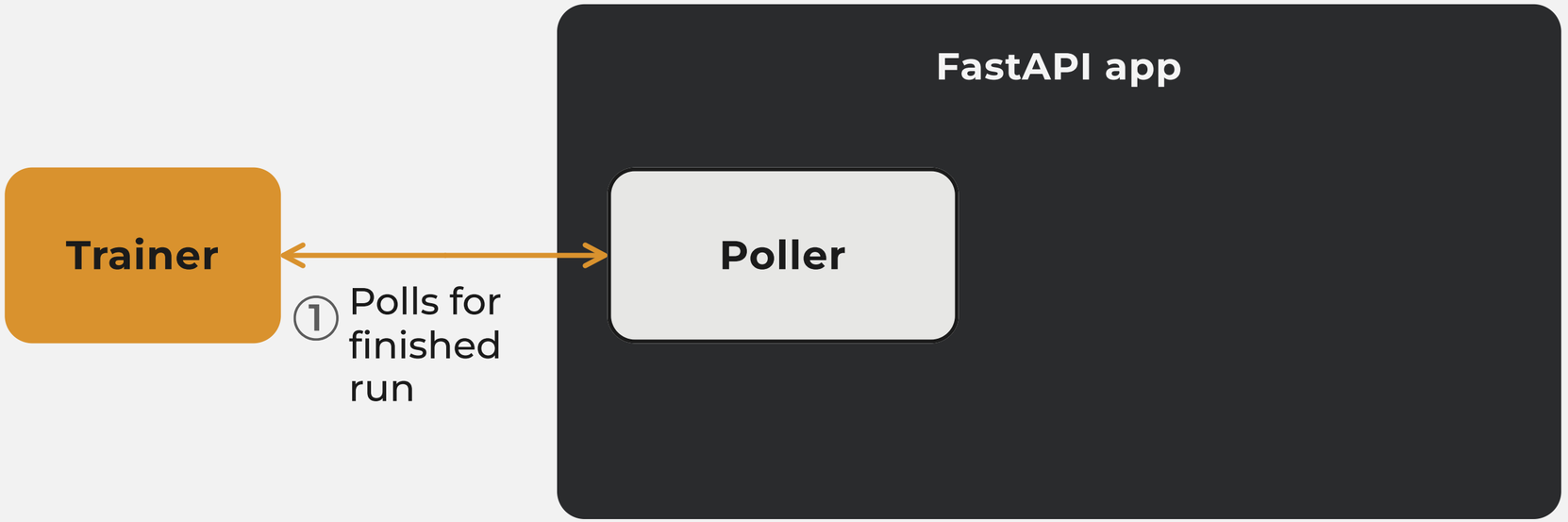
3 **Train a challenger**
In a separate container

Trainer

AUTO-RETRAINER

From Trained to Live

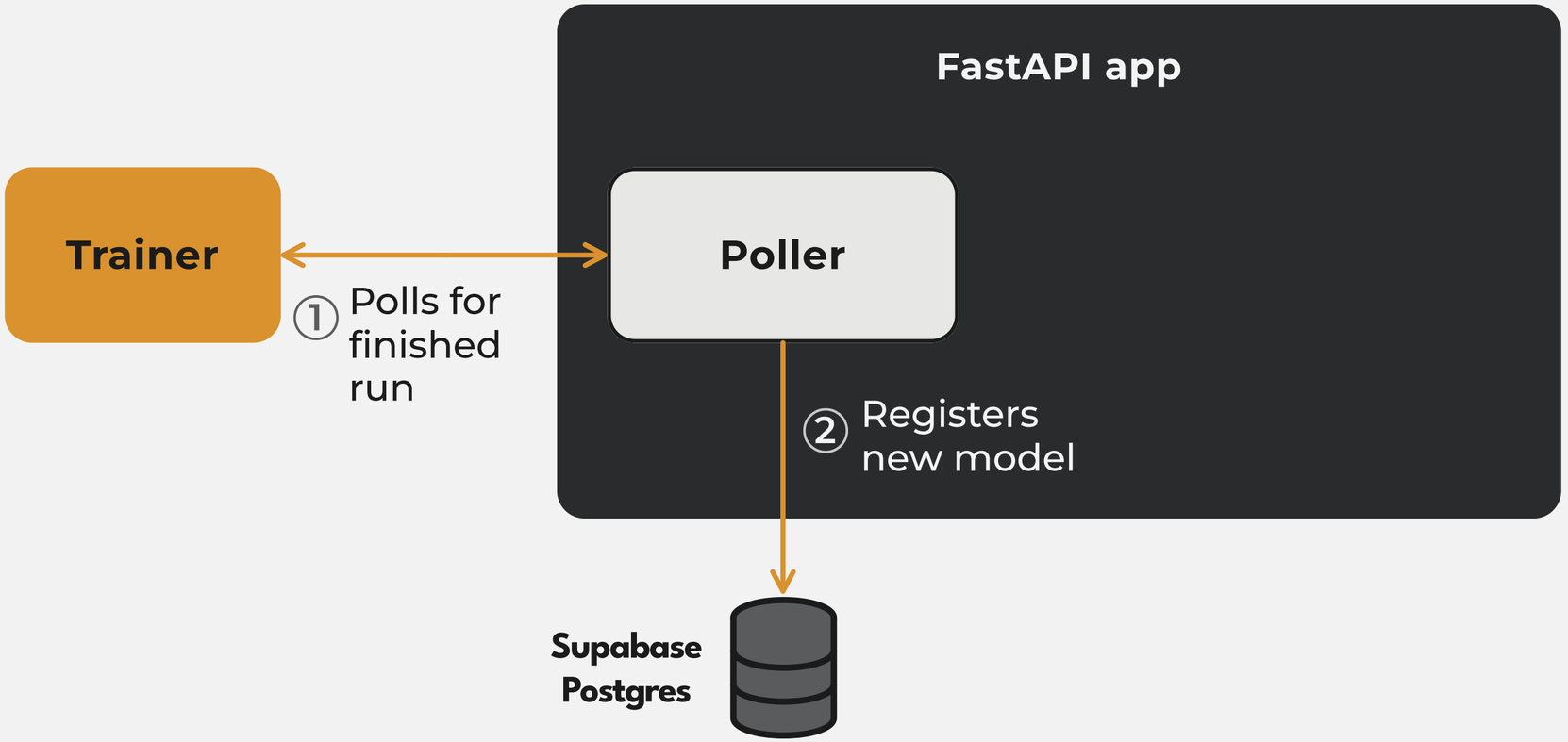
3 **Train a challenger**
In a separate container



AUTO-RETRAINER

From Trained to Live

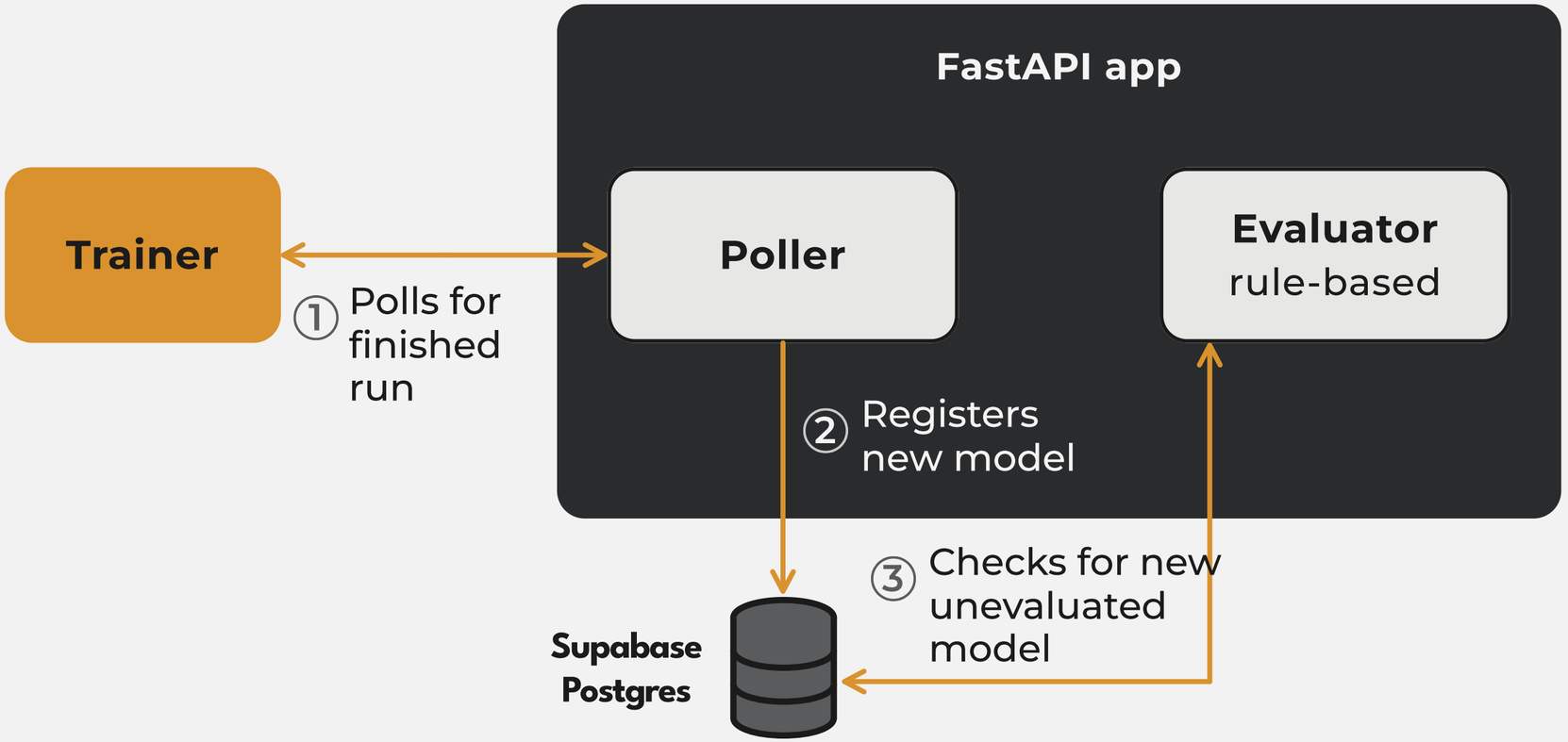
3 **Train a challenger**
In a separate container



AUTO-RETRAINER

From Trained to Live

3 **Train a challenger**
In a separate container

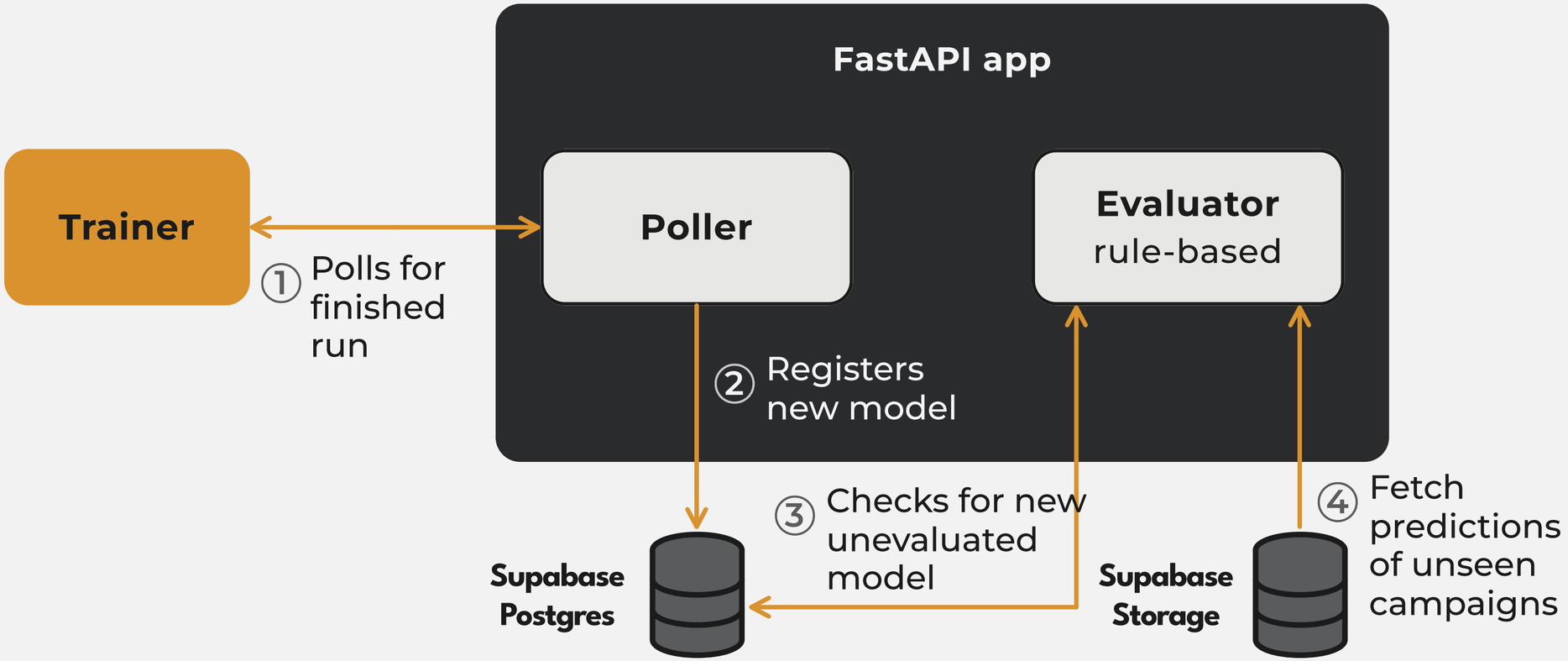


AUTO-RETRAINER

From Trained to Live

3 **Train a challenger**
In a separate container

4 **Grade it**
Against the current champion

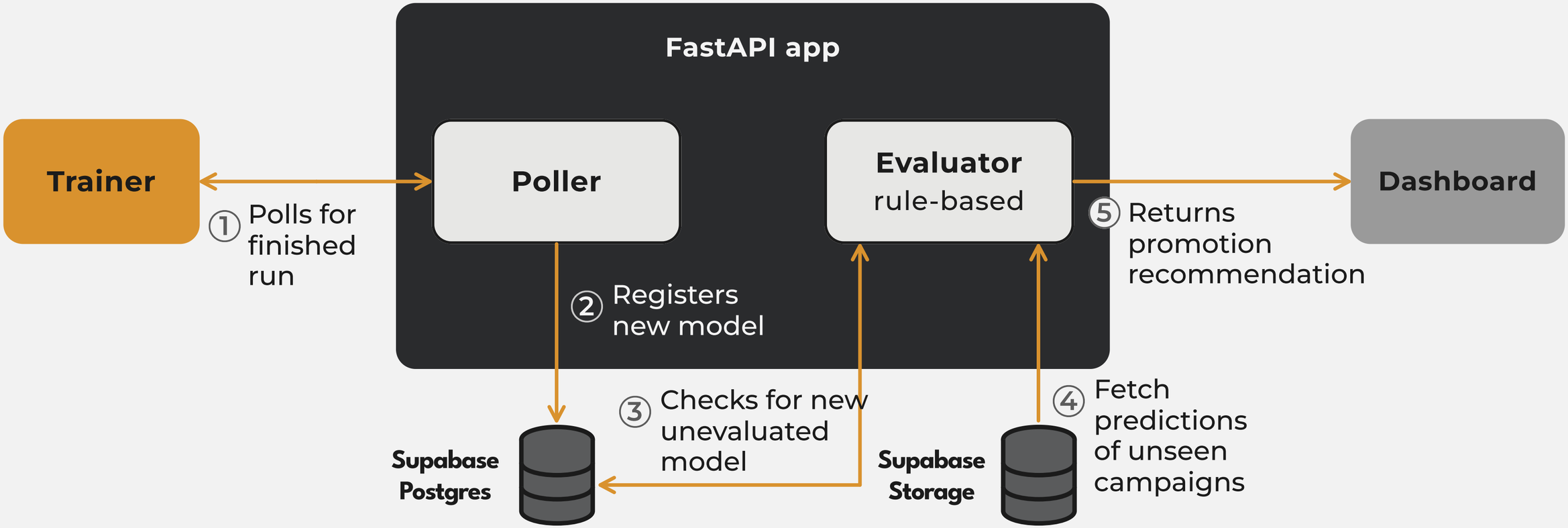


AUTO-RETRAINER

From Trained to Live

3 **Train a challenger**
In a separate container

4 **Grade it**
Against the current champion



AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test
3. **Train a challenger**
   In a separate container
4. **Grade it**
   Against the current champion

WHO DOES IT

- **Cassandra app**
- **Trainer**
- **Evaluator**

WHAT IT PRODUCES

- Trigger to start retraining
- Retraining dataset
- A challenger model
- Model promotion recommendation

AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test
3. **Train a challenger**
   In a separate container
4. **Grade it**
   Against the current champion
5. **C22 promotes challenger**
   On the dashboard

WHO DOES IT

Cassandra app	Trainer	Evaluator	Dashboard

WHAT IT PRODUCES

Trigger to start retraining	Retraining dataset	A challenger model	Model promotion recommendation	An active latest model

AUTO-RETRAINER

From Trained to Live

Dashboard
Promote?

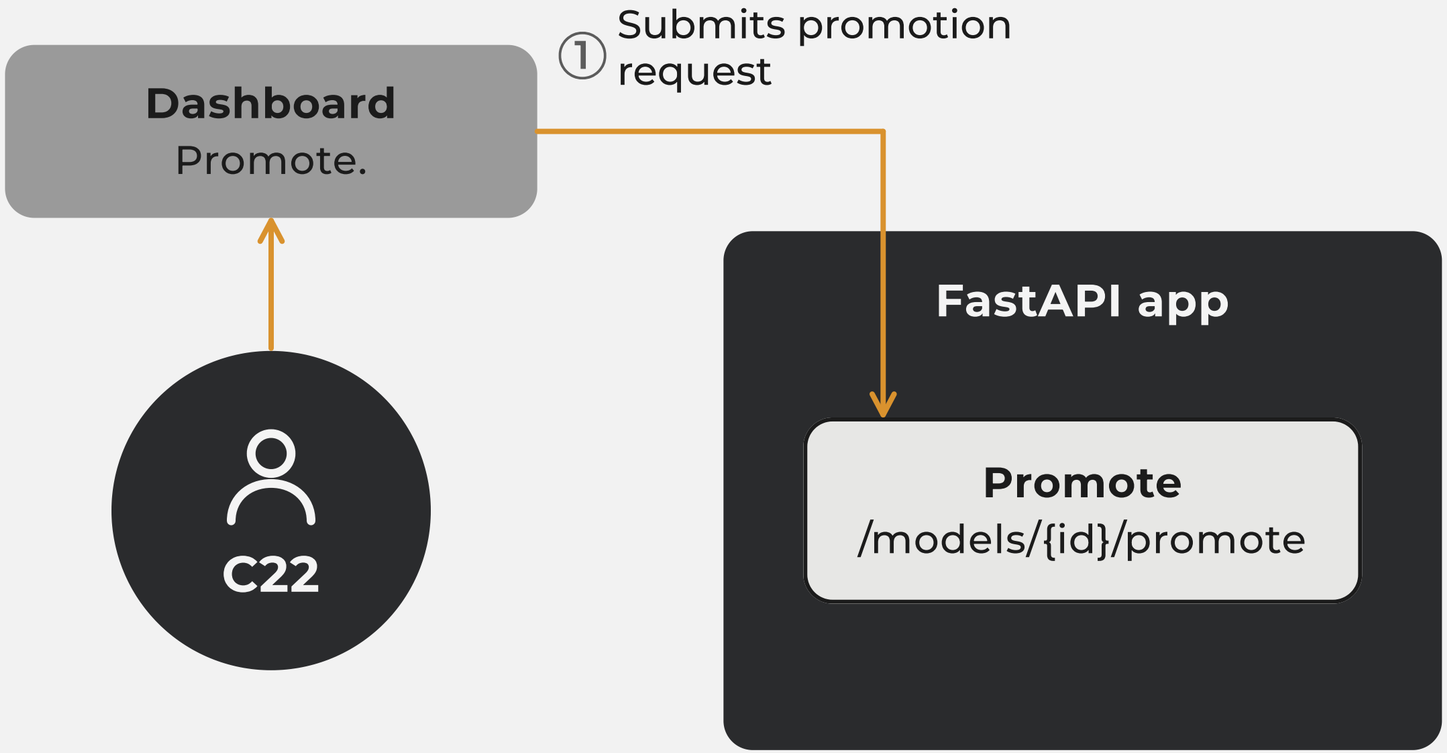


5 **C22 promotes challenger**
On the dashboard

AUTO-RETRAINER

From Trained to Live

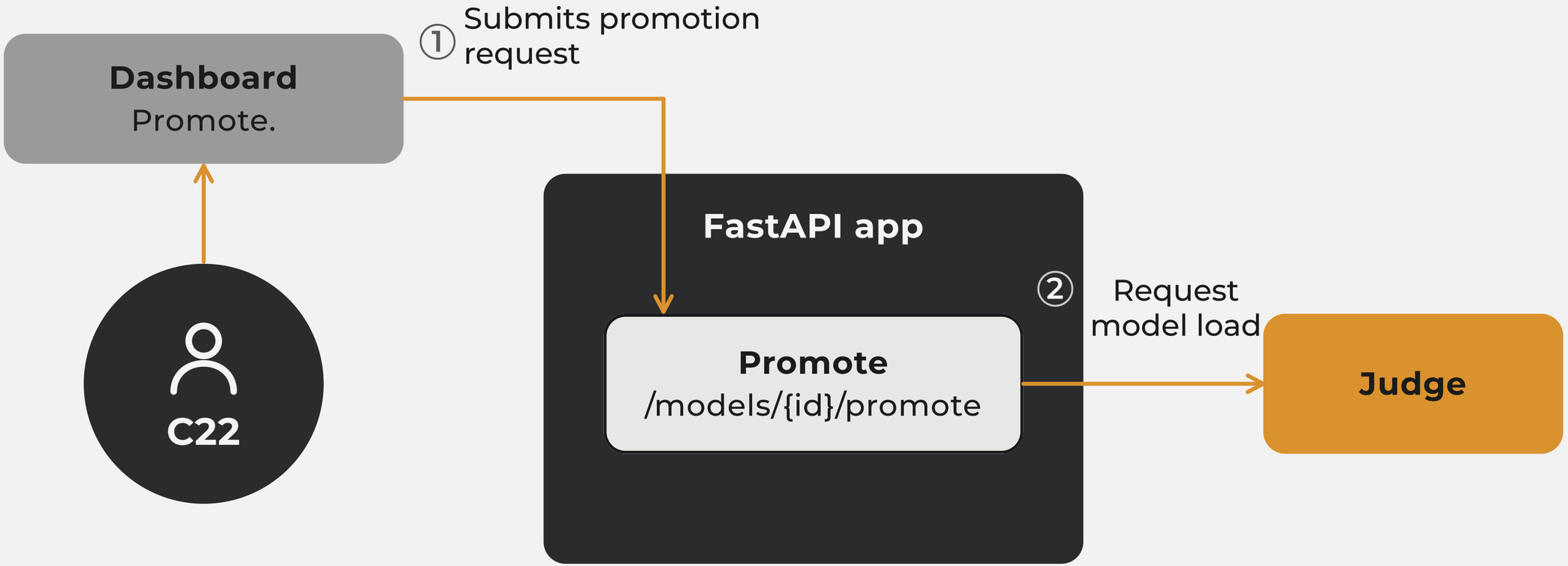
5 **C22 promotes challenger**
On the dashboard



AUTO-RETRAINER

From Trained to Live

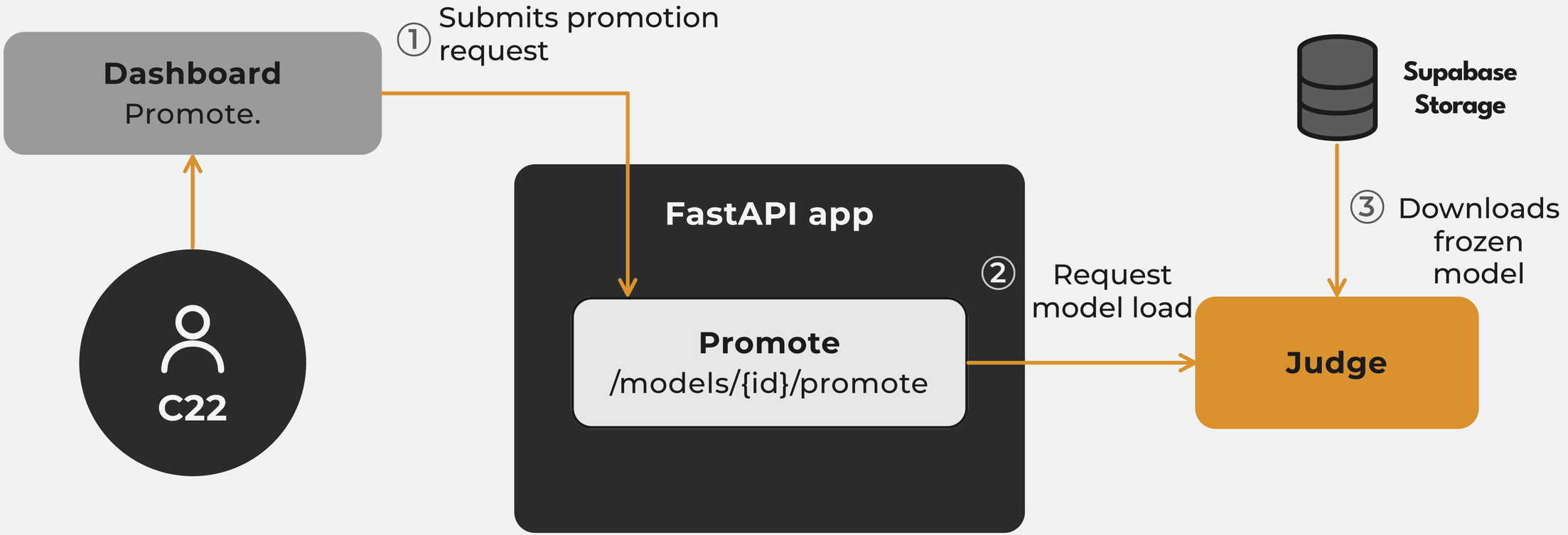
5 **C22 promotes challenger**
On the dashboard



AUTO-RETRAINER

From Trained to Live

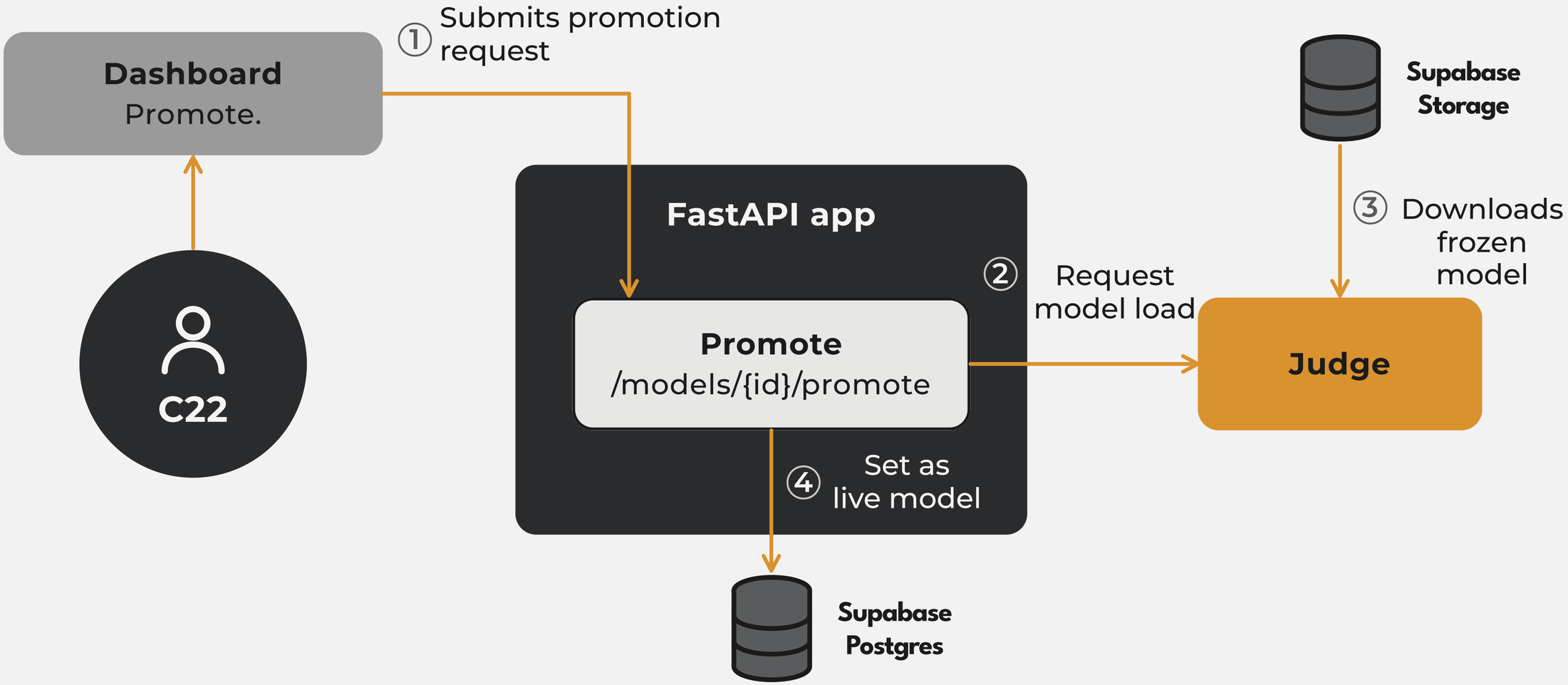
5 **C22 promotes challenger**
On the dashboard



AUTO-RETRAINER

From Trained to Live

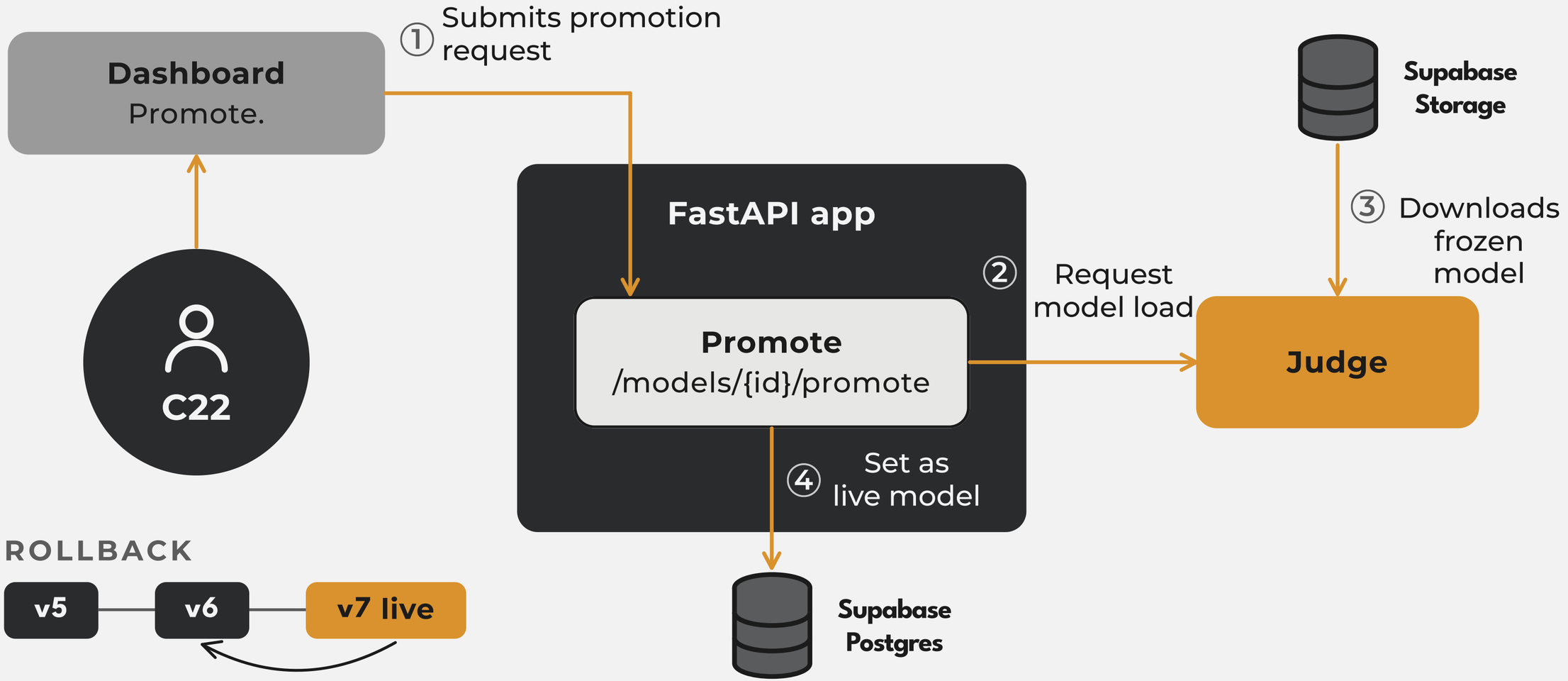
5 **C22 promotes challenger**
On the dashboard



AUTO-RETRAINER

From Trained to Live

5 **C22 promotes challenger**
On the dashboard



ROLLBACK

AUTO-RETRAINER

How Does a New Model Get Created

MAIN STAGES

1. **8 unseen campaigns**
   A run starts
2. **Prepare training data**
   Older 4 validate, newer 4 test
3. **Train a challenger**
   In a separate container
4. **Grade it**
   Against the current champion
5. **C22 promotes challenger**
   On the dashboard

WHO DOES IT

Cassandra app	Trainer	Evaluator	Dashboard

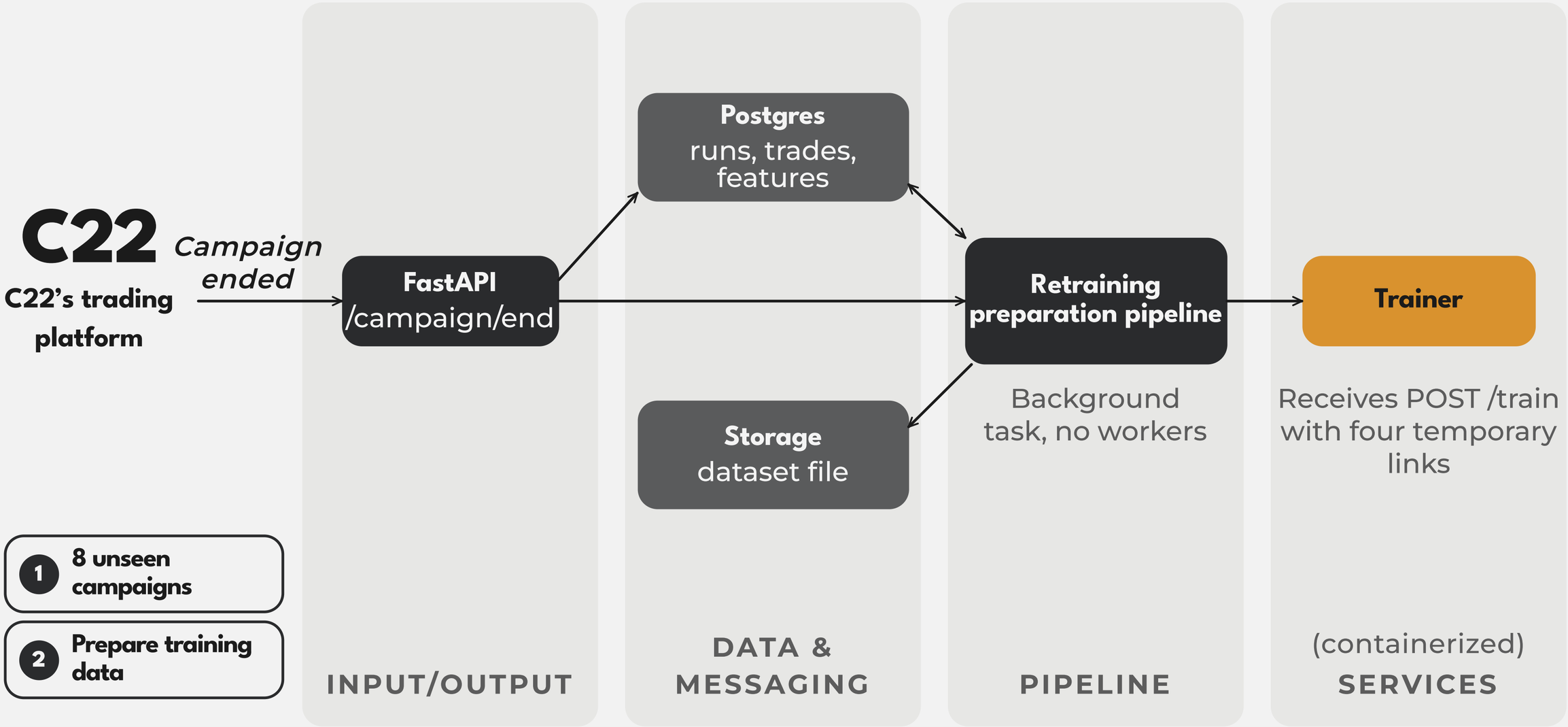
WHAT IT PRODUCES

Trigger to start retraining	Retraining dataset	A challenger model	Model promotion recommendation	An active latest model

The new model is promoted only when no campaign is live and C22 approves it.

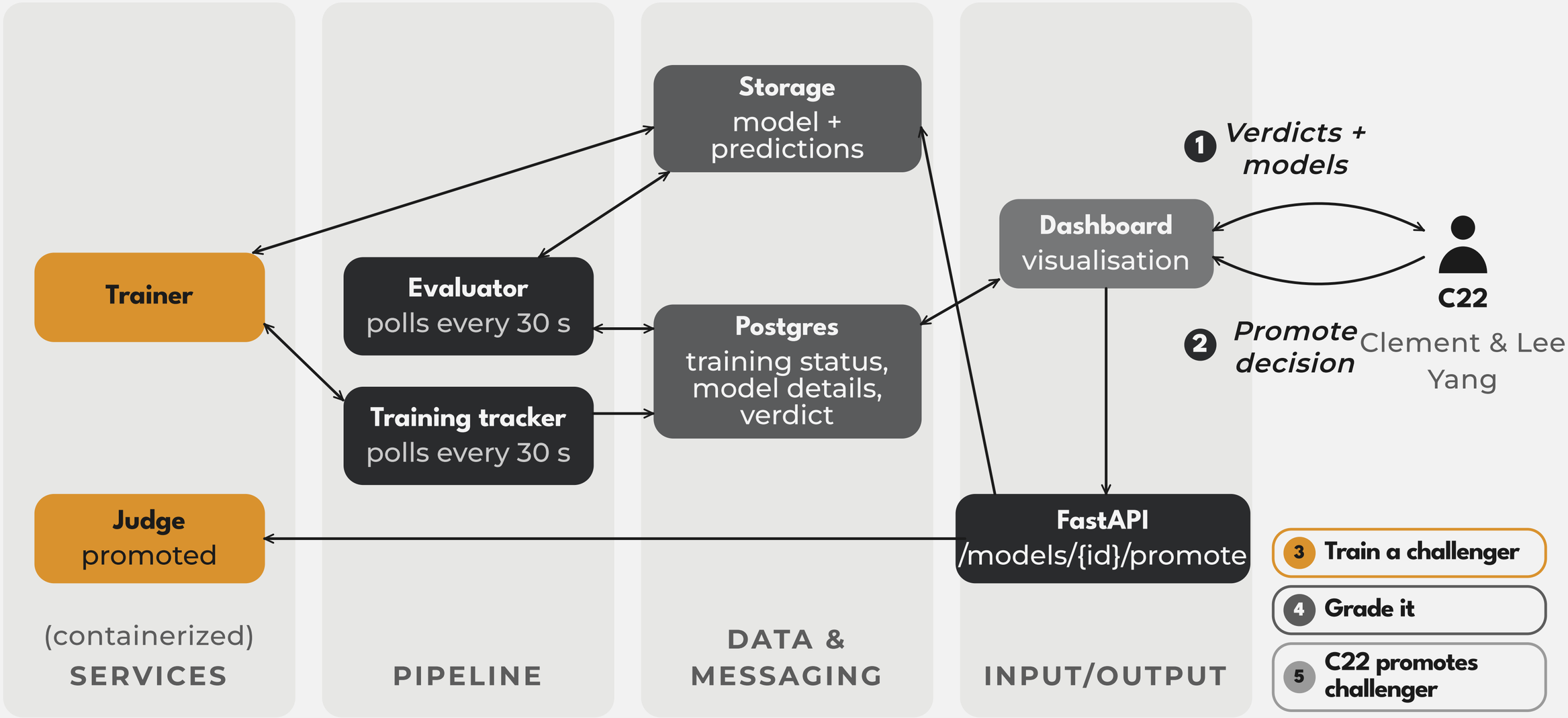
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



05

The Auto-Retrainer

From new campaigns to a promoted model

AUTO-RETRAINER CASSANDRA APP

Trigger and Split

When does retrainer decide to train a new model?

~1 month’s worth

Unseen campaigns since last retraining.

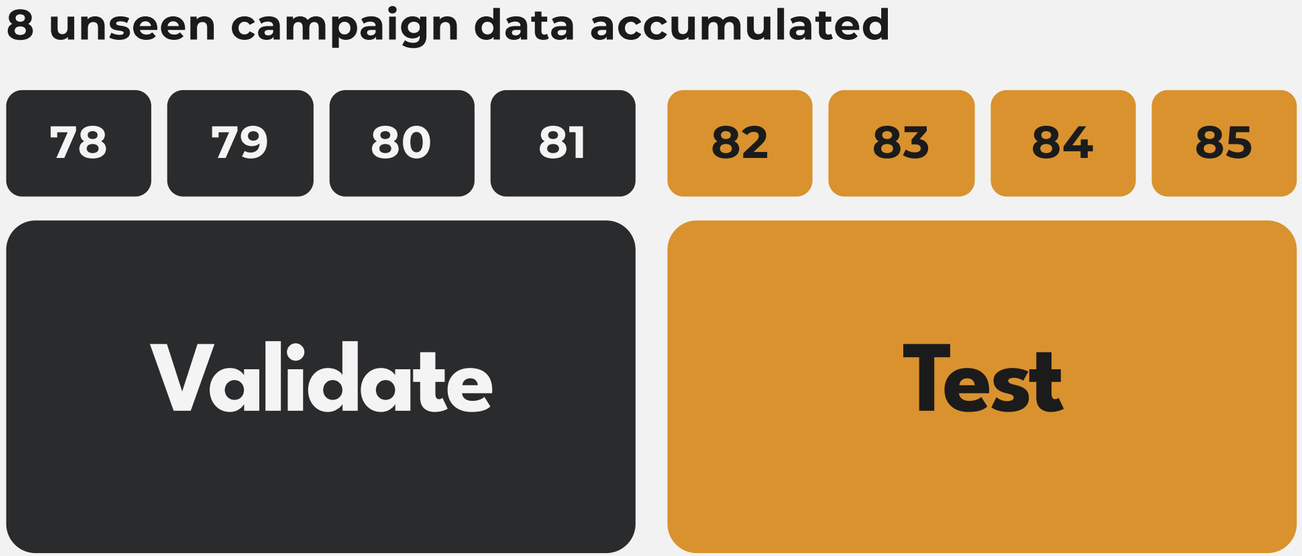
AUTO-RETRAINER CASSANDRA APP

Trigger and Split

When does retrainer decide to train a new model?

~1 month’s worth

Unseen campaigns since last retraining.



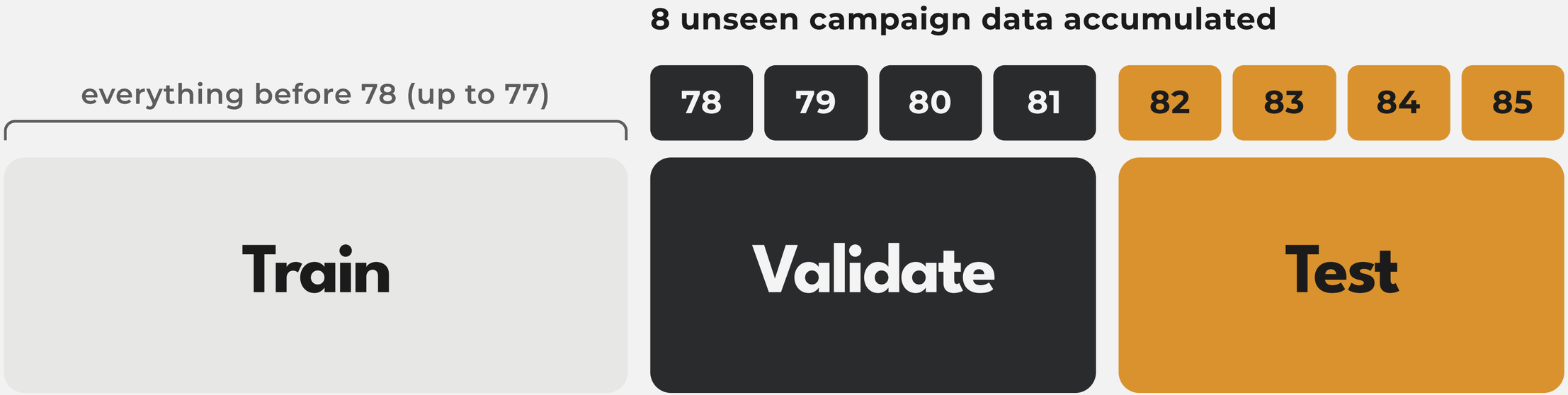
AUTO-RETRAINER CASSANDRA APP

Trigger and Split

When does retrainer decide to train a new model?

~1 month’s worth

Unseen campaigns since last retraining.



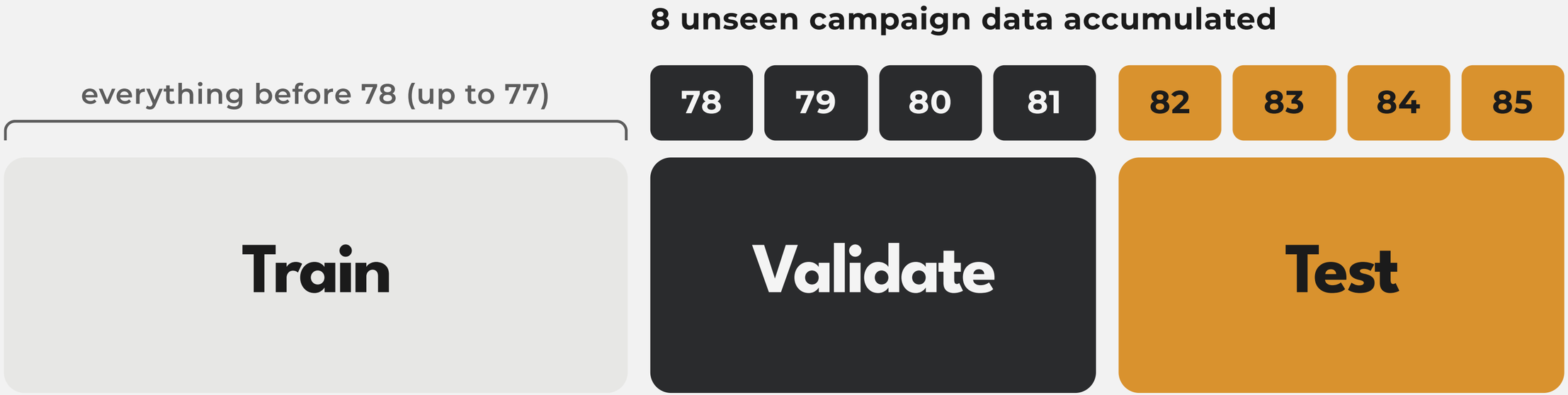
AUTO-RETRAINER CASSANDRA APP

Trigger and Split

When does retrainer decide to train a new model?

~1 month’s worth

Unseen campaigns since last retraining.



**Trained in multiples of 8 (e.g. if there are currently 17 campaigns worth of data, only 16 will be used)*

KEY DESIGN DECISION 1 OF 3

Count-Triggered Retraining

DECISION

Accumulate 8 campaigns, then retrain automatically

REJECTED

Retraining after every campaign, by hand, or on a cron job

WHY

A run needs enough new data, and a fixed number lets the app start it on its own

TRADE-OFF

At two campaigns a week, a run comes about once a month

KEY DESIGN DECISION 2 OF 3

Separation of Retrainer



DECISION

Train in a separate, dedicated container

REJECTED

Training inside the app or the Judge service

WHY

The pieces change independently, and no dependencies are duplicated

TRADE-OFF

One more container to manage

KEY DESIGN DECISION 3 OF 3

C22 Endorsed Promotions



DECISION

The evaluator recommends. C22 decides.

REJECTED

Promoting automatically after a good evaluation

WHY

The new model affects future campaigns, so a person should make the call

TRADE-OFF

It needs manual action, and a localised deployment sends no alerts

AUTO-RETRAINER · EVALUATOR

Four Checks, in Order

C22 can modify any threshold on the dashboard

KEY TERMS

AUC ranks winners above losers; 0.5 = coin flip

Brier how far predicted win probabilities are from results; lower is better

1 INPUTS

Can we trust the numbers?

Right data?

Unseen campaigns, each trade once, real live model

Model cheating?

Leakage gate: no feature tied to the outcome above 0.02

Numbers add up?

Recomputed totals match the trainer's report

Fail → Hold

2 LEAK ALARM

Is AUC suspiciously high?

AUC > 0.65

Signals a leak rather than a breakthrough

Fail → Hold

AUTO-RETRAINER · EVALUATOR

Four Checks, in Order

C22 can modify any threshold on the dashboard

KEY TERMS

AUC ranks winners above losers; 0.5 = coin flip

Brier how far predicted win probabilities are from results; lower is better

3 ON ITS OWN

Is the new model sound?

Ranks trades well?

Overall AUC ≥ 0.53, clearly above a coin flip

In every campaign?

Never below a coin flip (AUC 0.5)

Win probabilities accurate?

Brier lower than always guessing the average

Better than random?

Top-5% P&L beats random picks at least as well as the live model's

Fail → Keep

4 COMPARE

Better than the live model?

Earns more?

Top-5% P&L higher on average than the live model

Not just luck?

Top-5% P&L higher in over half the test campaigns

Nothing worse?

AUC and Brier not clearly worse than the live model

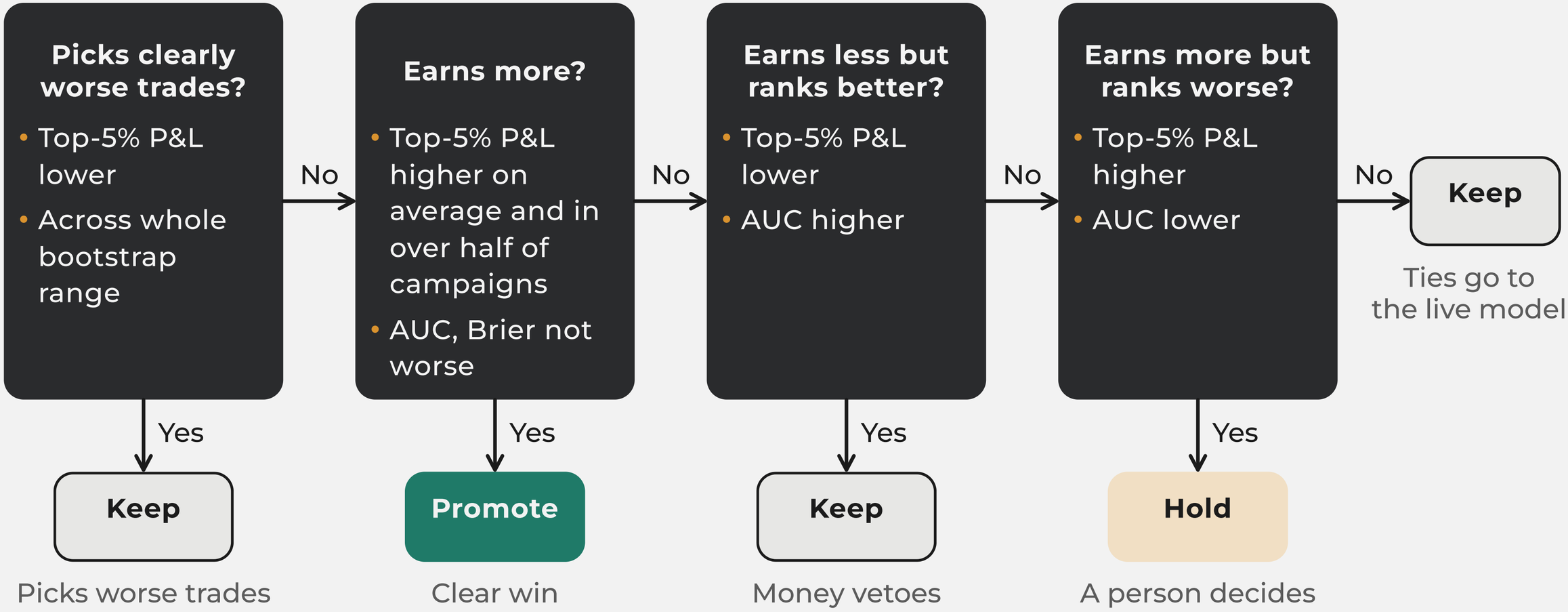
Promote · Keep · Hold

AUTO-RETRAINER · EVALUATOR

How Step 4 Decides

FAIR COMPARISON

Both models scored on the same unseen campaigns



06

Model Training

A quick recap of the Judge

SYSTEM ARCHITECTURE

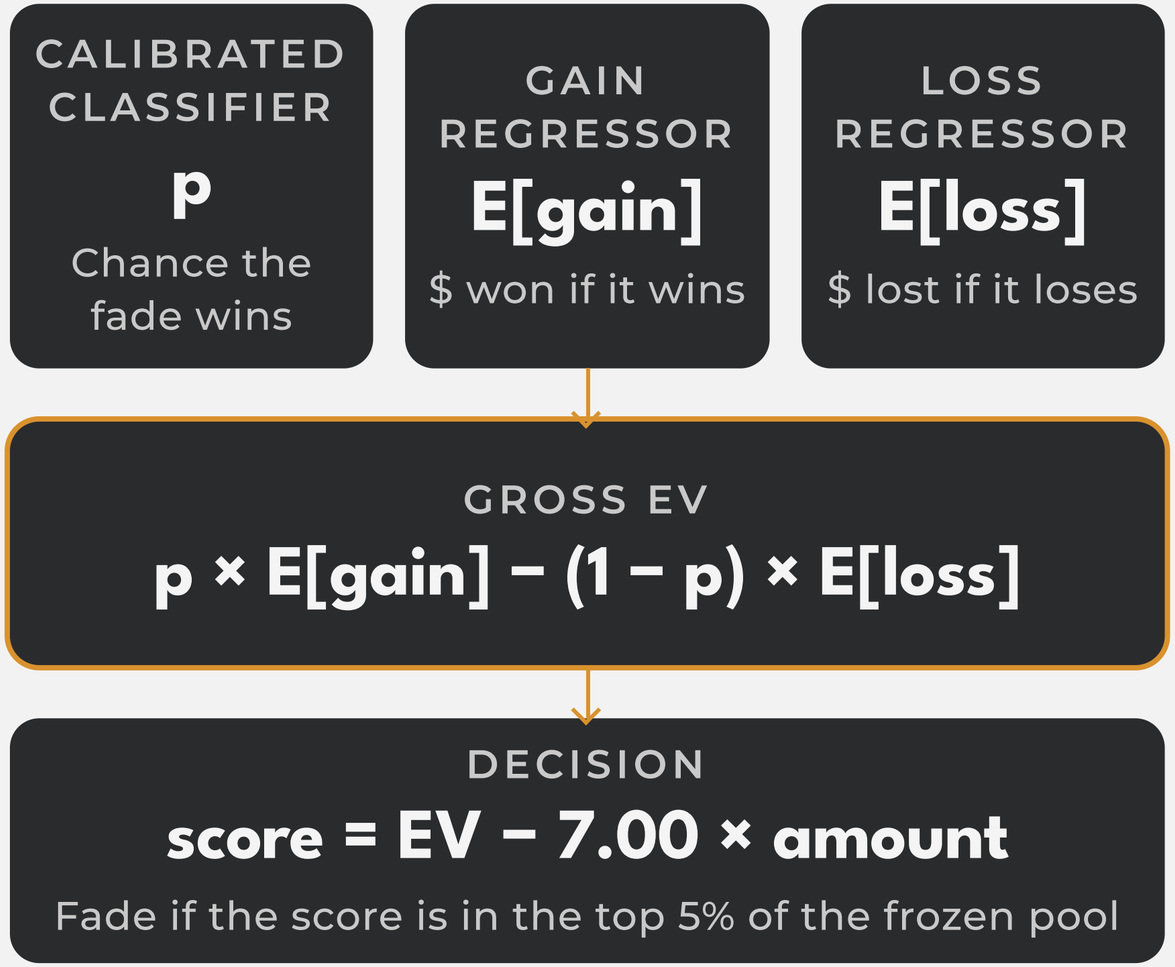
An ensemble of LightGBM models

In one sentence

Three small models. Each fills in one term of the expected-value equation.

Why it fits

- **Tabular data**
- **Small dataset**
- **Built-in regularisation**
- **Fast inference**



STEP 01: DATA

How the Data was used

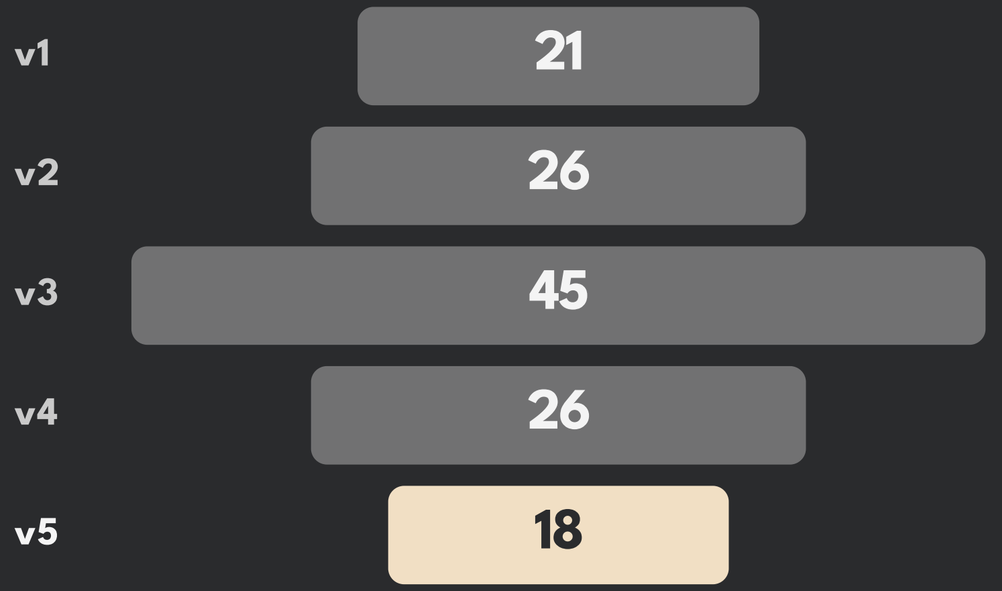
54

Campaigns from C22

Every customer trade is a row of 28 fields with information on trader's side, amount, open and close prices, and open and close times

Campaigns	
33–68	Training set
69–77	Testing set
78–86	Additional test set

How the feature count changed



Our final feature set, frozen for judge model

STEP 02: FEATURE ENGINEERING

The 18 Features We Kept

6

The trade itself

Side, amount, hour given; others derived

- **Buy or sell** `side_is_buy`
- **Trade size** `amount`
- **Hour opened** `hour_utc`
- **Time on a clock face** `tod_sin, tod_cos`
- **Gold's move today** `price_vs_campaign_open_bps`

8

Earlier today

From the customer's finished trades today

- **Trades so far today** `n_prior_trades`
- **Size vs their usual** `escalation_ratio`
- **Usual hold time** `mean_prior_duration`
- **Take-profit habit** `prior_tp_use_rate`
- **First trade of the day** `is_cold_start`
- **Wait since last trade** `sec_since_last_close`
- **Stop-loss habit** `prior_sl_use_rate`
- **Usual stop distance** `mean_prior_sl_dist_bps`

4

Earlier days

From campaigns already finished

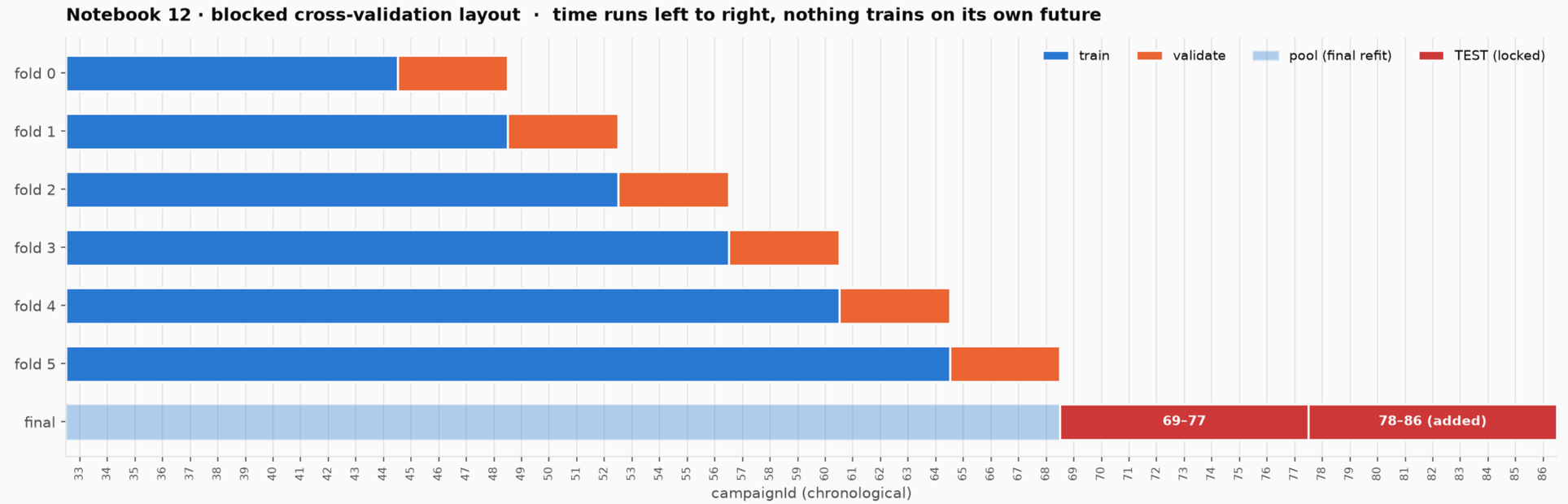
- **Past days traded** `n_prior_campaigns`
- **Trades on past days** `n_prior_campaign_trades`
- **Days since last traded** `days_since_last_campaign`
- **Past stop-loss habit** `prior_campaign_sl_use_rate`

STEPS 03–04: TRAINING & VALIDATION

How We Cross-Validate

BLOCKED CV · CAMPAIGNS 33–86

Six expanding folds, then a locked test block



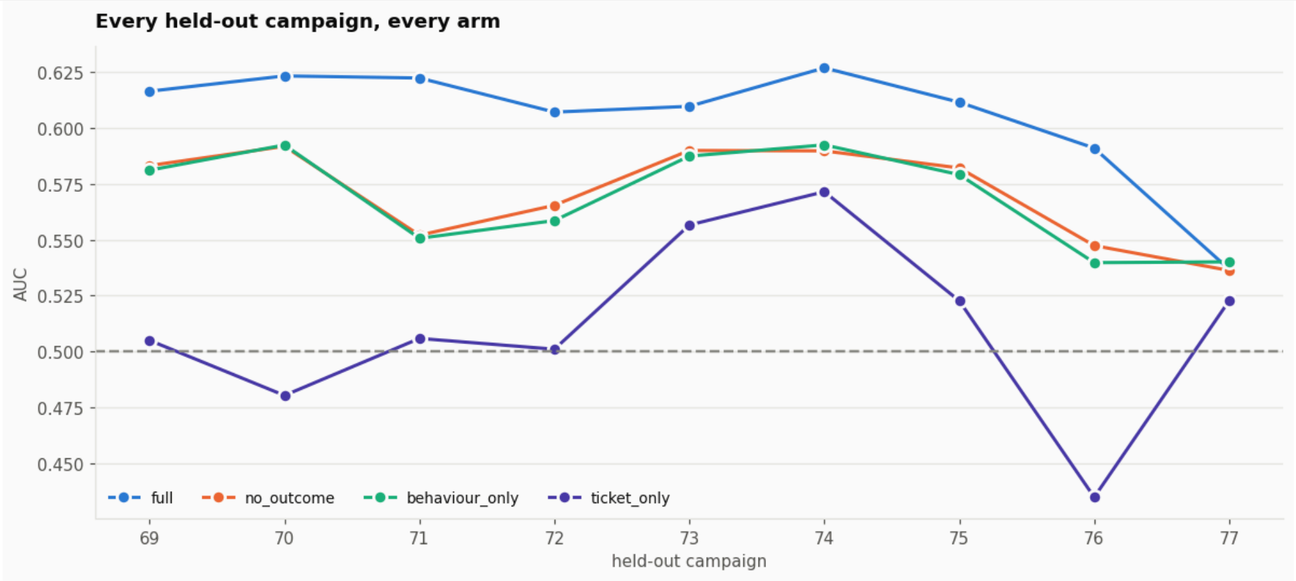
Time runs left to right
Nothing trains on its own future

STEP 04: VALIDATION

Every Arm, Every Campaign

HELD-OUT AUC · CAMPAIGNS 69–77

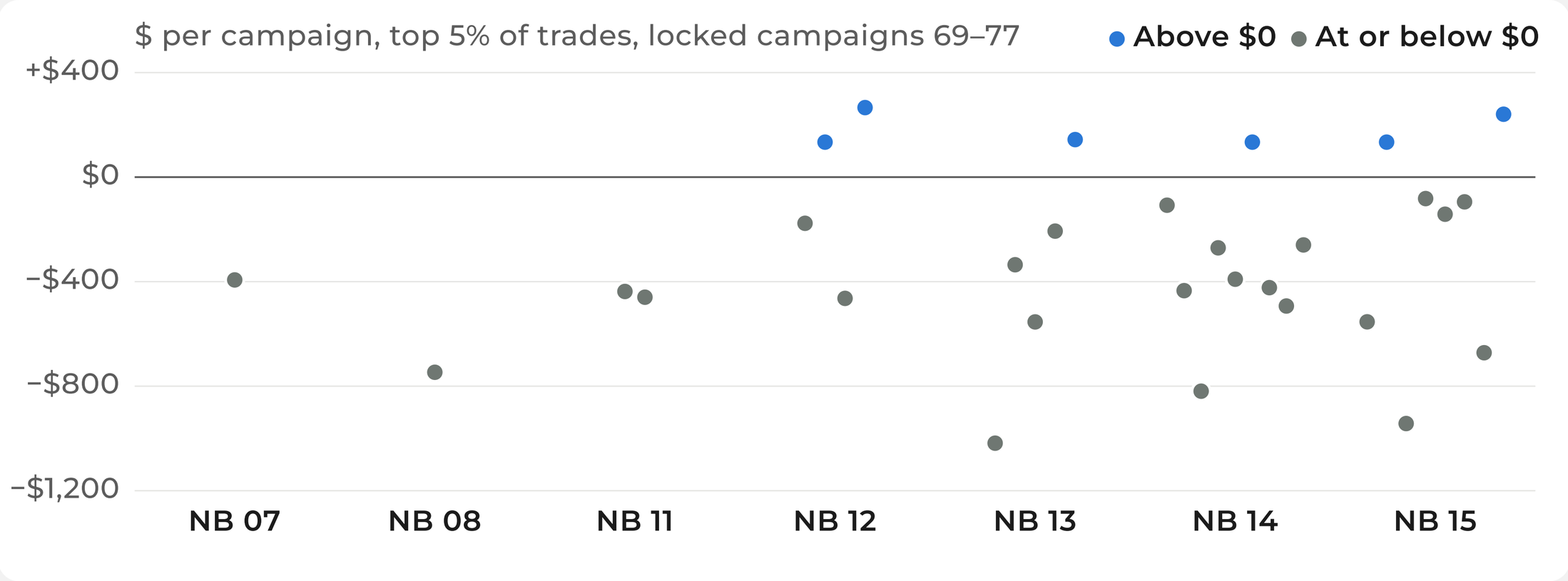
Feature-set ablation



Removing outcome history hurts in 9 of 9 campaigns
Ticket size alone sits near chance

STEP 04: VALIDATION

What Each Notebook Tried



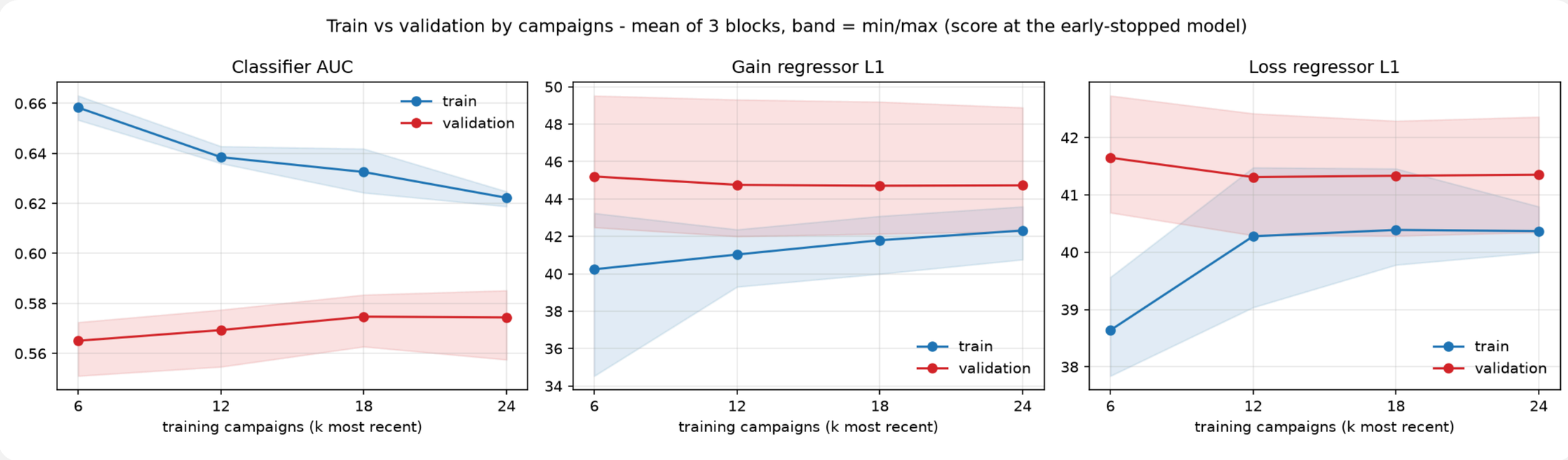
Only 6 of 30 arms finish above $0.
Where we have intervals (NB 13–15), every one spans zero.

STEP 04: VALIDATION

Train vs Validation by Campaigns

MEAN OF 3 BLOCKS · BAND = MIN/MAX

More campaigns shrink the gap, not the error



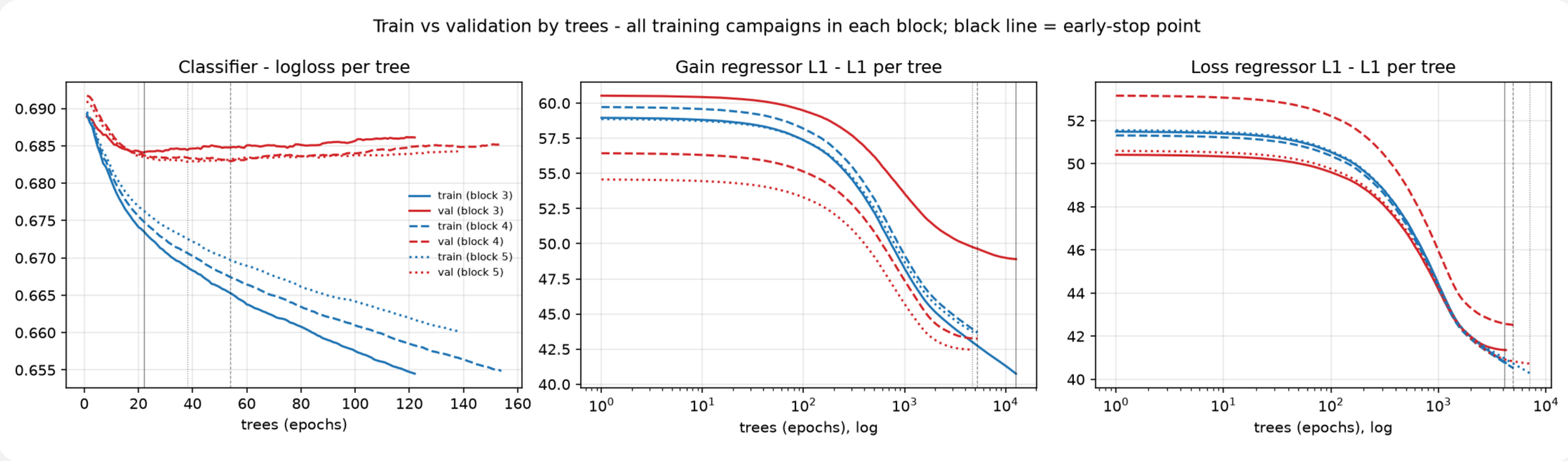
Classifier gap narrows (AUC 0.66 → 0.62 train, 0.565 → 0.575 val); regressor L1 stays flat

STEP 04: VALIDATION

Train vs Validation by Trees

BLOCKS 3–5 · BLACK LINE = EARLY STOP

The classifier overfits fast; the regressors do not



Classifier stops after 22–54 trees; regressors keep improving for thousands

07

Explanations

Why the Judge fades a trade, in plain English

EXPLANATIONS

How It Works

1. **The Judge scores the trade in dollars**
2. **TreeSHAP splits that score across features**
3. **Features roll up into six families**
   size, pace, stops, experience, timing, market
4. **Fixed templates write the top 2–4 as sentences**

Not an LLM

Same trade, same words, every time

Never in the decision

The explanation reads the score. It does not change it

EXPLANATIONS

An Example

Trade 288230376154358301

The trade, the Judge's decision, and the reasons it gave.

Trader Campaign Account ID D#1702473-75	Side SELL
Amount 0.20	Open Date Time 05 Aug 2026, 08:42:05
Status COMPLETED	Reverse Profit -23.00
Judge Decision SKIP	Recommended Side SELL

Why the Judge decided this

Stops Against fade ≈ −$8
Stops: set a stop-loss on 20% of their earlier trades in this campaign (pool p37).

Size Against fade ≈ −$4
Size: 0.20 lots (pool p64).

Pace Against fade ≈ −$4
Pace: opened 4.6 h after their previous trade in this campaign closed (pool p98).

Describes what moved the Judge's score. It is not evidence that the trade will pay.

Trade 288230376154355126

The trade, the Judge's decision, and the reasons it gave.

Trader Campaign Account ID D#1702553-75	Side SELL
Amount 0.25	Open Date Time 05 Aug 2026, 06:47:01
Status COMPLETED	Reverse Profit 27.00
Judge Decision FADE	Recommended Side BUY

Why the Judge decided this

Experience Toward fade ≈ +$2
Experience: 0 earlier closed trades in this campaign (pool p9).

Timing Toward fade ≈ +$1
Timing: opened in the 22:00 UTC hour (0.3% of pool trades open in this hour).

Describes what moved the Judge's score. It is not evidence that the trade will pay.

EXPLANATIONS

Readable by Anyone

BEFORE

A dollar score

+$26.06. Only a data person can say why.

NOW

A plain sentence

Size: 0.50 lots (pool p95).
Experience: 0 earlier closed trades (pool p9).

WHY IT MATTERS

Trust in the Judge

Staff can follow a decision without data skills.

EXPLANATIONS

Themes After a Campaign

GROUP

Reasons across trades

Every fade's reasons are grouped by family over the campaign.

READ

Size leads on 64%

Locked block, 659 fades: size 425, stops 143, experience 82, side 9.

WHY IT MATTERS

Faster review

Read a few themes, not thousands of trades.

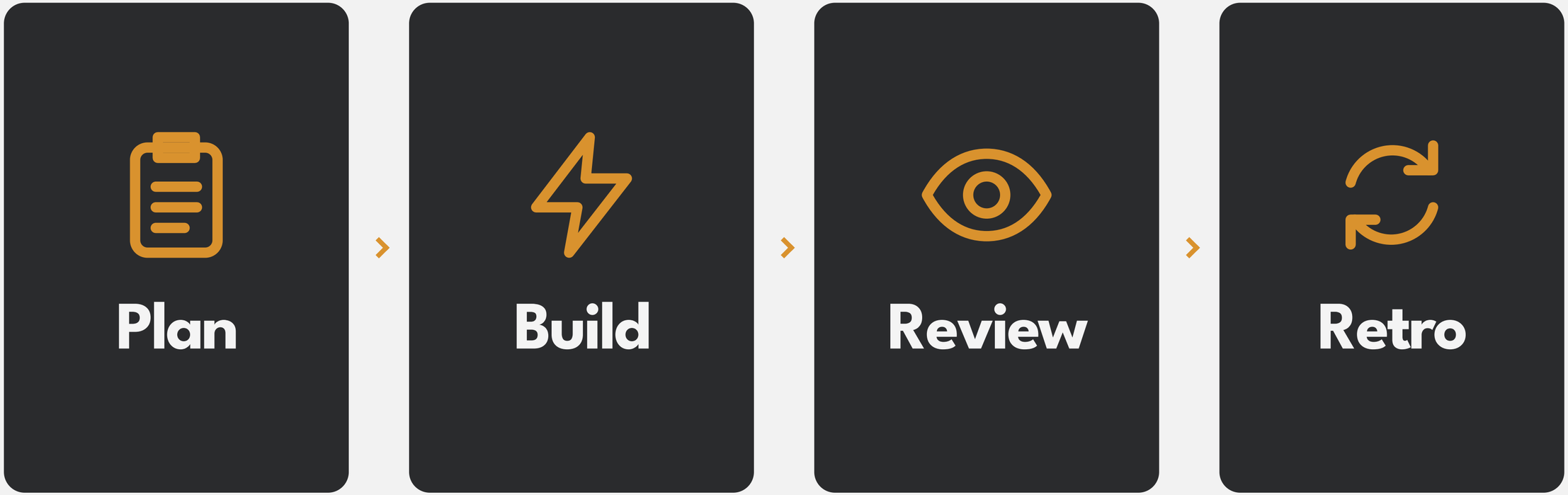
08

Project Management

How we ran the work

PROJECT MANAGEMENT

How We Ran Scrum



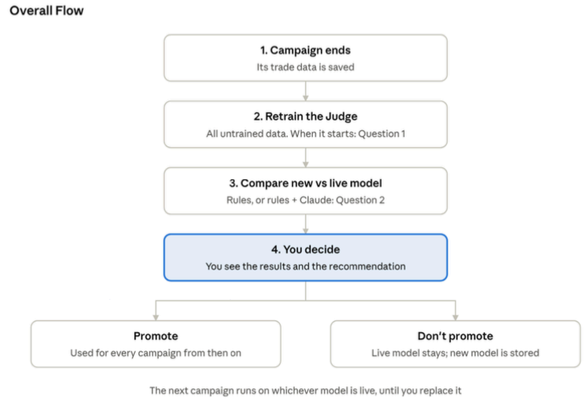
PROJECT MANAGEMENT

Feedback Changes the Backlog

Judge retraining and promotion: two decisions

How it works

After a campaign ends, a new Judge model is trained, compared with the live one and offered to you. Your two decisions are when step 2 starts and how step 3 makes its recommendation.



Retraining and promotion flow · 4 steps, 2 outcomes

- Each retrain uses all campaign data the Judge has not been trained on yet.
- Models you don't promote are stored, not deleted.
- **Note:** Cassandra must stay on while a retrain runs. If it is switched off midway, that retrain is lost and has to start again. No new campaign can start until it finishes.

Question 1: When should retraining start? (step 2)

Each retrain uses all campaign data the Judge has not been trained on yet, so how much data goes in depends on when it runs.

- **A. Fixed schedule (automatic).** Retraining starts by itself at an interval you set, for example after every campaign, every 3 campaigns or every month. There is no button. If you pick this, please tell us your preferred interval.
- **B. You decide (button).** The dashboard shows how much untrained data has built up. Retraining starts only when you press the button; until then, data keeps accumulating.

	A: Fixed schedule (automatic)	B: You decide (button)
What you do	Set the interval once	Press the button each time
New candidate models	Arrive on a predictable schedule	Arrive only when you press; none if you don't
Data per retrain	Set by the interval; a short interval adds little new data each time	Grows until you press
System busy	A few minutes after each interval	Only when you choose

Question 2: How should the promotion recommendation be made? (steps 3 and 4)

In both options you approve every promotion. The rules and Claude only recommend.

- **A. Rules only.** Fixed checks in code (table below) compare the new model with the live one. You get one recommendation, one reason and a report of every check.
- **B. Rules + Claude.** The rules still run first and still block any model that fails a check. Claude then ranks the live model, the new model and earlier stored models that passed. It recommends one, or keeping the live model, and explains why in plain English.

The rules run in three steps on test campaigns neither model has seen. Each step must pass before the next one runs.

Step	Checks	Recommendation if any fails
1. Can we trust the data?	Test campaigns unused in training; the live model scored is the one actually live; numbers add up to the cent; no feature leaks the answer	Check first

We use Scrum, so our backlog changes with what the client tells us.

PROJECT MANAGEMENT

Sprint 2 → Sprint 3

WHAT WE FOUND	WHAT WE CHANGED	WHAT IT LOOKED LIKE
Sprint 2 had no Wednesday check-in. BA worked async and nobody knew who did what	A live Wednesday sync every sprint, so we stay aligned	Sprint 3 retro: kept, and timeboxed to 15 to 20 min
Work was finished on the last day	Wednesday is the internal deadline, leaving two to three days of buffer	Sprint 3 retro: nothing left to put in Stop

09

Handover

What C22 receives, and what is left

HANDOVER

What C22 Receives

API	Trade ingestion, paper trading and campaign endpoints
Judge	Scores each trade FADE or SKIP
Trainer	Trains a challenger and grades it against the champion
Dashboard	Campaign, Judge performance, live trades, model evaluation
Database	Supabase schema
Docs	Operator guides, retraining contract, evaluator rules, developer guide

HANDOVER

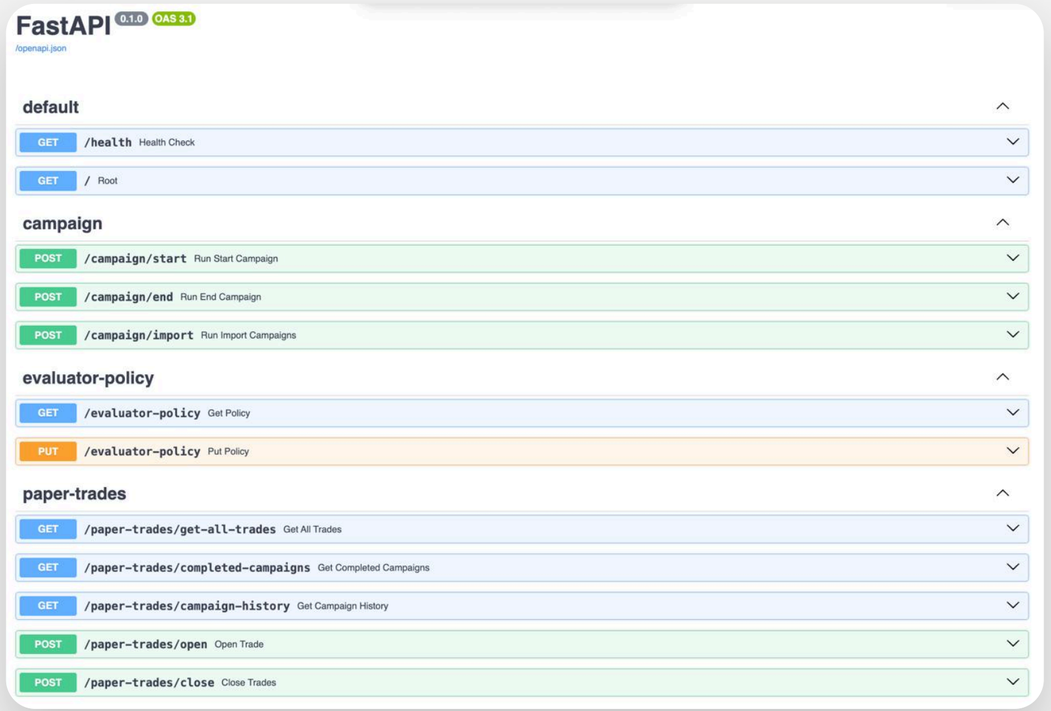
What Is Left

One thing is left: connecting C22's platform to our system.



C22's platform is not ready yet

✓ API interfaces documented cleanly for C22 to connect



10

Conclusion

What we verified, explored and built

CONCLUSION · 1 OF 3

Help Inform C22's Next Steps

> **Some of your findings line up with what I've been seeing independently. Crowd followers don't lose more often. Matches my result that the crowd tells you about price. I think the model itself yall work on is pretty good, is just maybe like you said the lack of data that can't paint a better picture.**

OUR SPONSOR, C22

Research partnership: we checked their hypothesis independently

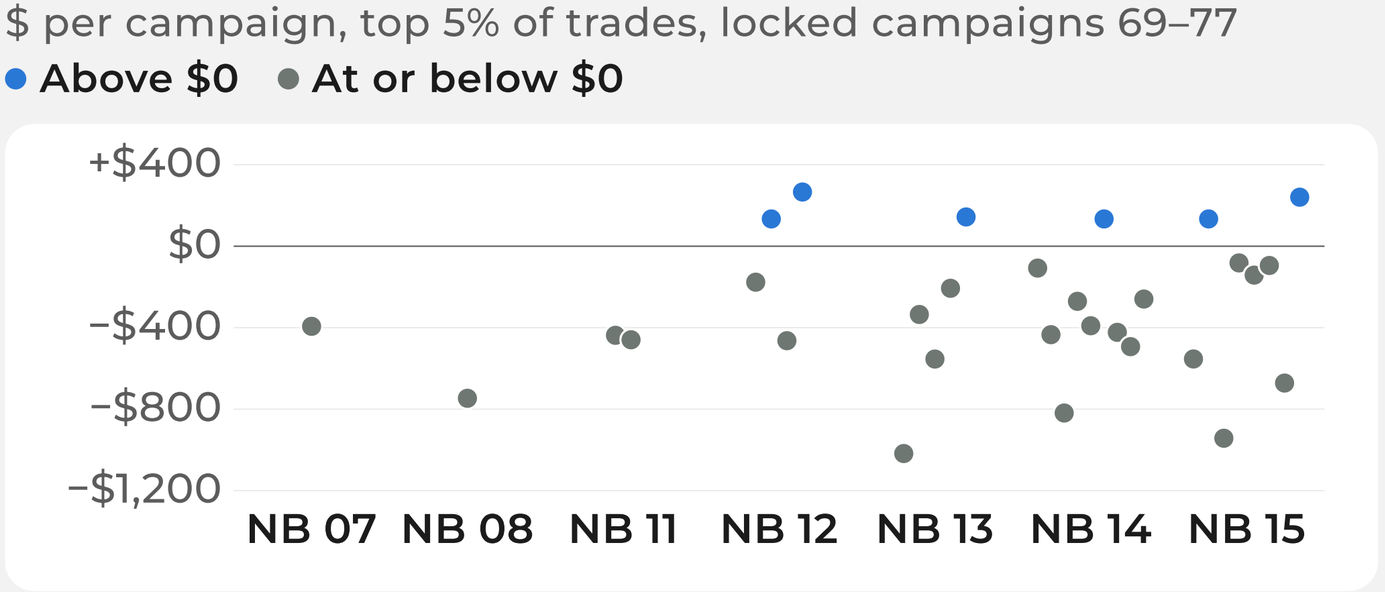
CONCLUSION · 2 OF 3

A Fresh Perspective

30

Variants explored

Feature sets, model types and targets, across notebooks 7 to 15.



Fresh perspective: new eyes on features and models

CONCLUSION · 3 OF 3

Creating a System That Outlives Us



Creating a System That Outlives Us



To update the model, Clement imports a file.

CONCLUSION · 3 OF 3

Creating a System That Outlives Us



Retraining starts automatically.

CONCLUSION · 3 OF 3

Creating a System That Outlives Us



Graded automatically. Promoted by a person.

A system that outlives us

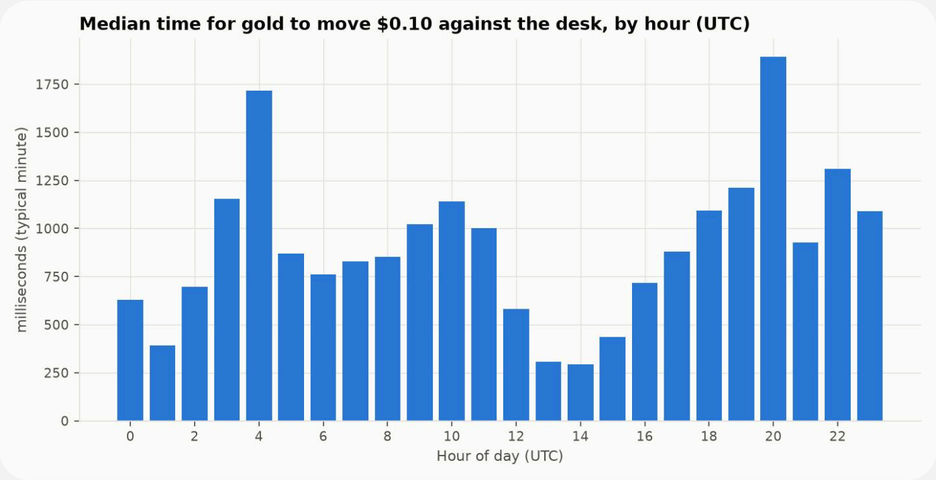
Thank you

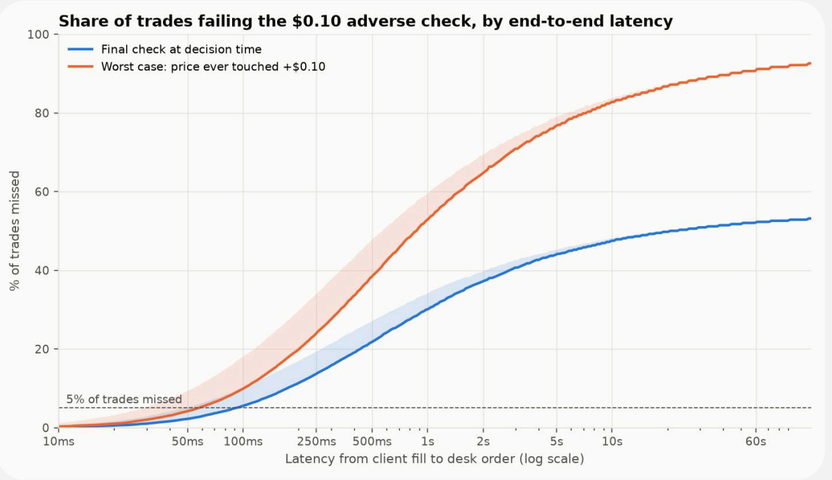
Appendix

Backup charts and tables for Q&A

APPENDIX · NON-FUNCTIONAL REQUIREMENTS

Latency and Throughput





Latency how fast gold moves

- **Median move time:** 0.6–0.85 s, about 0.3 s in the 13–14 UTC London/New York overlap.
- **Typical minute:** a $2.15 range, about 21× the 10c threshold.
- **Latency budget:** ≤5% of trades fail the check at ~50–90 ms; ≤10% at ~100–170 ms; ≤25% at ~0.4–0.6 s.

Throughput 84,456 trades, Feb–Sep

- **Peak:** 5.9 trades/min (14 Aug, 09:30–10:00).
- **Bursts:** busiest minute 19 trades; busiest second 5, from 5 accounts. Median gap between trades is 24 s.
- **Build for:** bursts of 5–10 simultaneous trades. Only 2% of trades close within 5 s.

APPENDIX · NON-FUNCTIONAL REQUIREMENTS

Latency Budget vs Measured

What the data says 10c adverse-price check

- **Under ~100 ms:** the check only works if fill → desk order is this fast. At 1 s, ~30% of trades already fail.
- **Strict budget:** ≤1% failing needs ~15–30 ms.
- **Calm vs volatile:** 5% failing allows ~0.7 s in a calm minute, ~3 ms in the most volatile 1% (news spikes).
- **Stricter reading:** if any move to +10c before the order counts, limits roughly halve.
- **Data limit:** nothing finer than 1 s, so sub-second is scaled. The real market is slower there, so these limits are cautious.

Our peak run

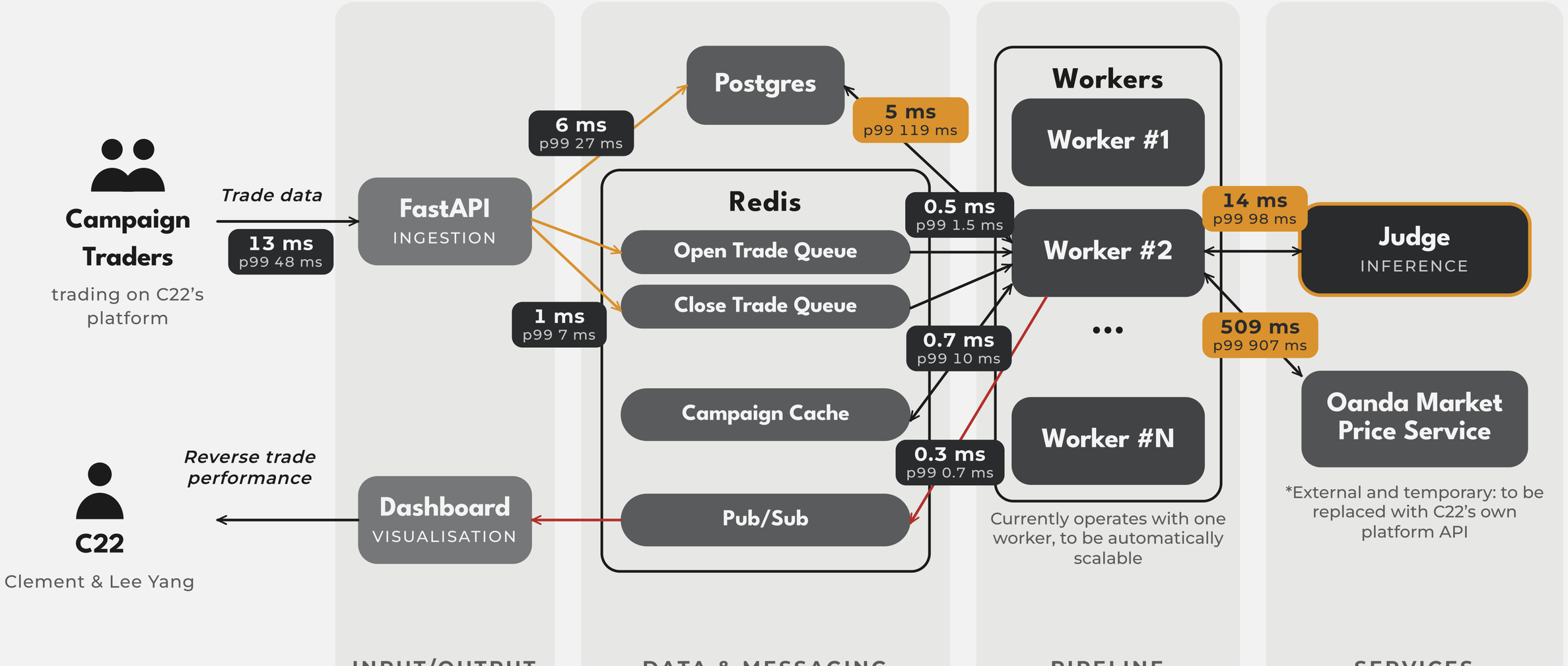
Milliseconds from API receipt. 102 trades: 99 completed, 3 slipped. About 10% pass 100 ms; the desk order waits on OANDA.

	p50	p90	p99	Max
Decision	**31**	88	228	241
OANDA read	**558**	746	951	1,850

APPENDIX · NON-FUNCTIONAL REQUIREMENTS

Where the Time Goes

p50 of 102 open trades, p99 beneath. The clock starts when the API receives the trade.



APPENDIX · NON-FUNCTIONAL REQUIREMENTS

The Slow 10%

What slows the tail p50 → p99

- **Judge call:** 10 → 87 ms. A new HTTP client is built every call (80 ms at p99). Share one.
- **Counting closed trades:** 2 → 107 ms. One DB query with a bad tail. Check with EXPLAIN, likely a missing index.
- **Storing the open trade:** 8 → 31 ms. Two commits where one would do.

Not the problem ruled out

- **Queue wait:** 0.5 ms (p99 1.5 ms). With 15 workers, one is always free.
- **Redis steps:** under 1 ms at the median.
- **Judge model:** about 3 ms.
- **Bursts of 5:** no slowdown.

The real problem is OANDA

If the desk order waits for the ~558 ms price read, about 100% of trades miss the budget.

SYSTEM ARCHITECTURE

Workflow 1: Campaign Start

1. **Snapshot the market at start time**
2. **Close any previous campaign still running**
3. **Create campaign and coinflip seed**
4. **Register the traders for current campaign**
5. **Compute each trader's history**
6. **Cache snapshot and histories in Redis**

Fetch reference price

Used in price movement since start of campaign, calculated for each trade

Six features per trader

Prior campaigns | Trades | Win rate | Profit | Days since last campaign | Stop-loss use

No history: zeros and nulls

Steps 1–6 run in one database transaction. If any step fails, nothing is saved and a 500 response is sent with failed step

SYSTEM ARCHITECTURE

Workflow 2: Open Trade

1. **Trade arrives**
   Trader opens a new position
2. **Store trade in Postgres, queue it**
   Picked up by workers running the open-trade pipeline

ASYNC · OPEN TRADE PIPELINE RUN BY WORKER

3. **Read cached trader history from Redis**
4. **Build 18 features, call the Judge**
   Inference returns a decision from the features
5. **Fetch market price, check slippage**
6. **Record the simulated reverse position**
   Market price + Judge decision, stored for reverse profit calculation

Key notes

- Market data will be polled or read from C22 directly, with no response sent.
- Slippage threshold is 0.10 as set by C22, adjusted for this demo because of historical data.

Market price

Currently using the Oanda API as a placeholder; will integrate with C22

Slippage threshold

Max allowed difference in market price between when the trader opened their position and when we open ours

SYSTEM ARCHITECTURE

Workflow 3: Close Trade

1. **Close trade arrives**
   Trader closes an existing position
2. **Store trade in Postgres, queue it**
   Picked up by workers to run the close-trade pipeline

ASYNC · CLOSE TRADE PIPELINE RUN BY WORKER

3. **Update running stats in Redis**
   Campaign and trader stats, for subsequent inferences
4. **Fetch market price at close time**
5. **Calculate simulated profit/loss**
   Based on the Judge's decision
6. **Update campaign's live totals**
   Published to the dashboard

Key notes

- Rolling metrics update in Redis cache even if the system fails to simulate a close position
- Rolling metrics are updated safely with a read-check-write retry pattern, not locks
- Live reverse trading PnL are streamed to the dashboard the moment each trade closes

SYSTEM ARCHITECTURE

Workflow 4: Campaign End & Evaluation

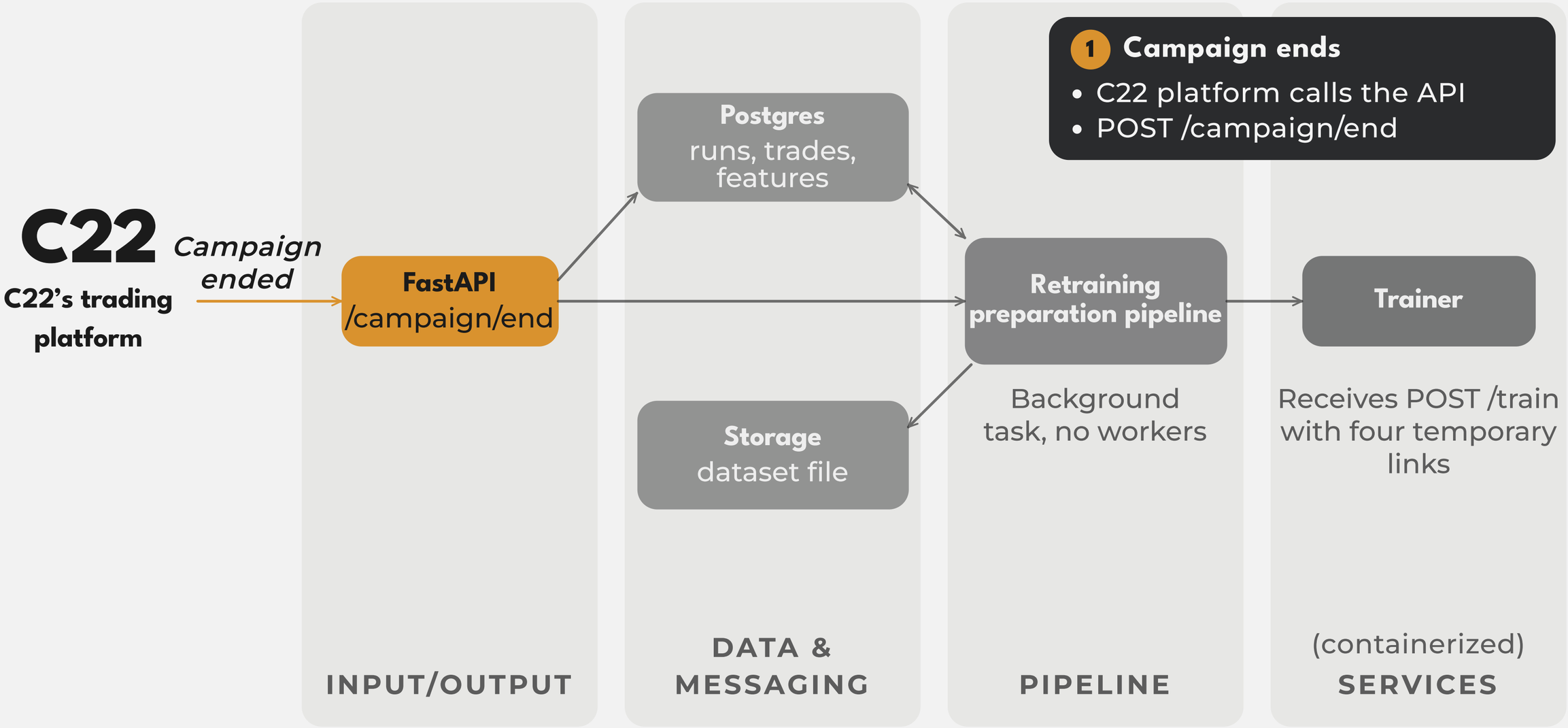
1. **Campaign end trigger received**
   Via API call to /end
2. **Campaign marked completed in Postgres**
3. **Any trade still open is force-failed**
4. **Aggregate values for 3 strategies**
   On completed trades: Judge, Reverse-all, Coin-flip
5. **Store results, clear campaign cache**

Key notes

- **Idempotent closing:** duplicated requests to end an already ended campaign is nullified and does not rerun the flow
- **Final PnL evaluation:** the following strategies are used on the same trade data and compared:
  - Judge decision-led reverse
  - Reverse all trades
  - Coin flip-led reverse

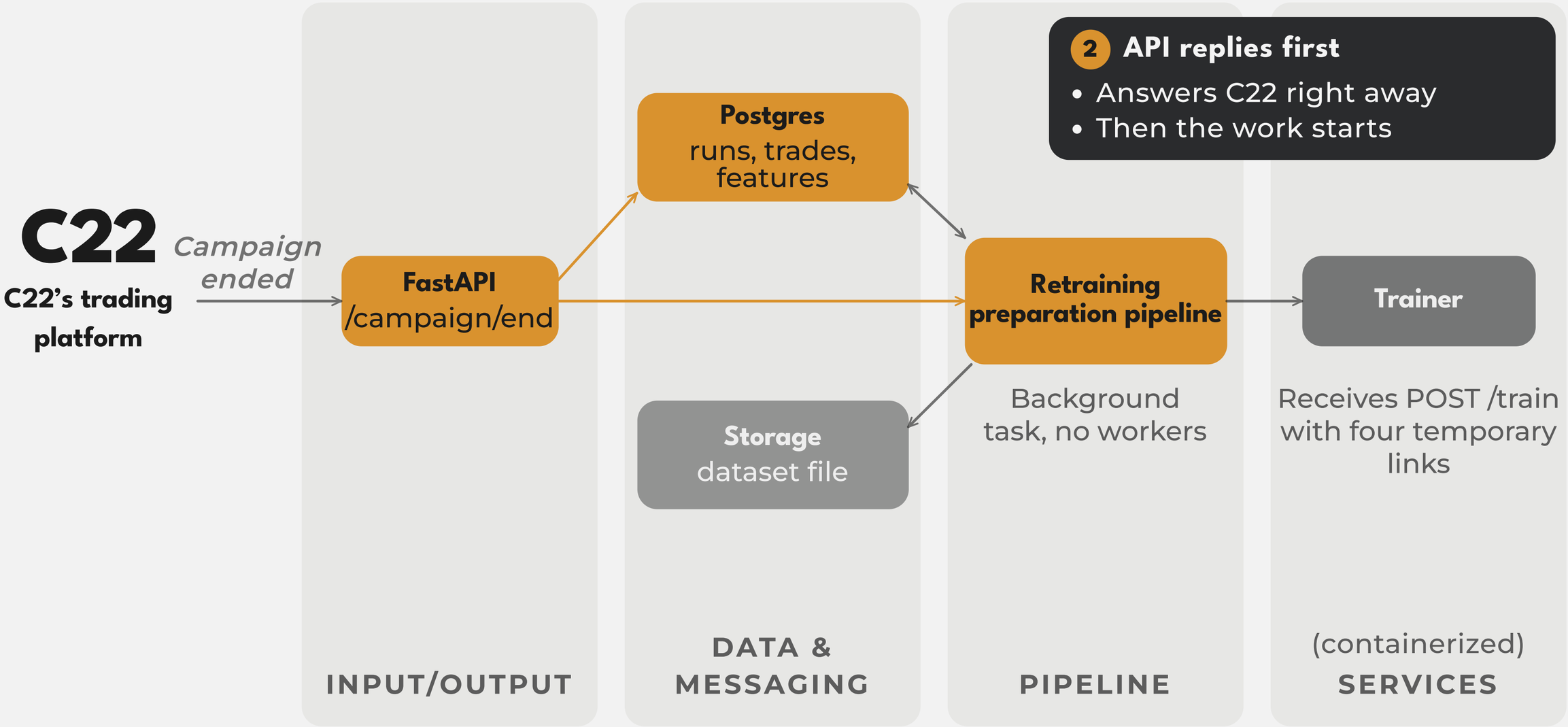
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



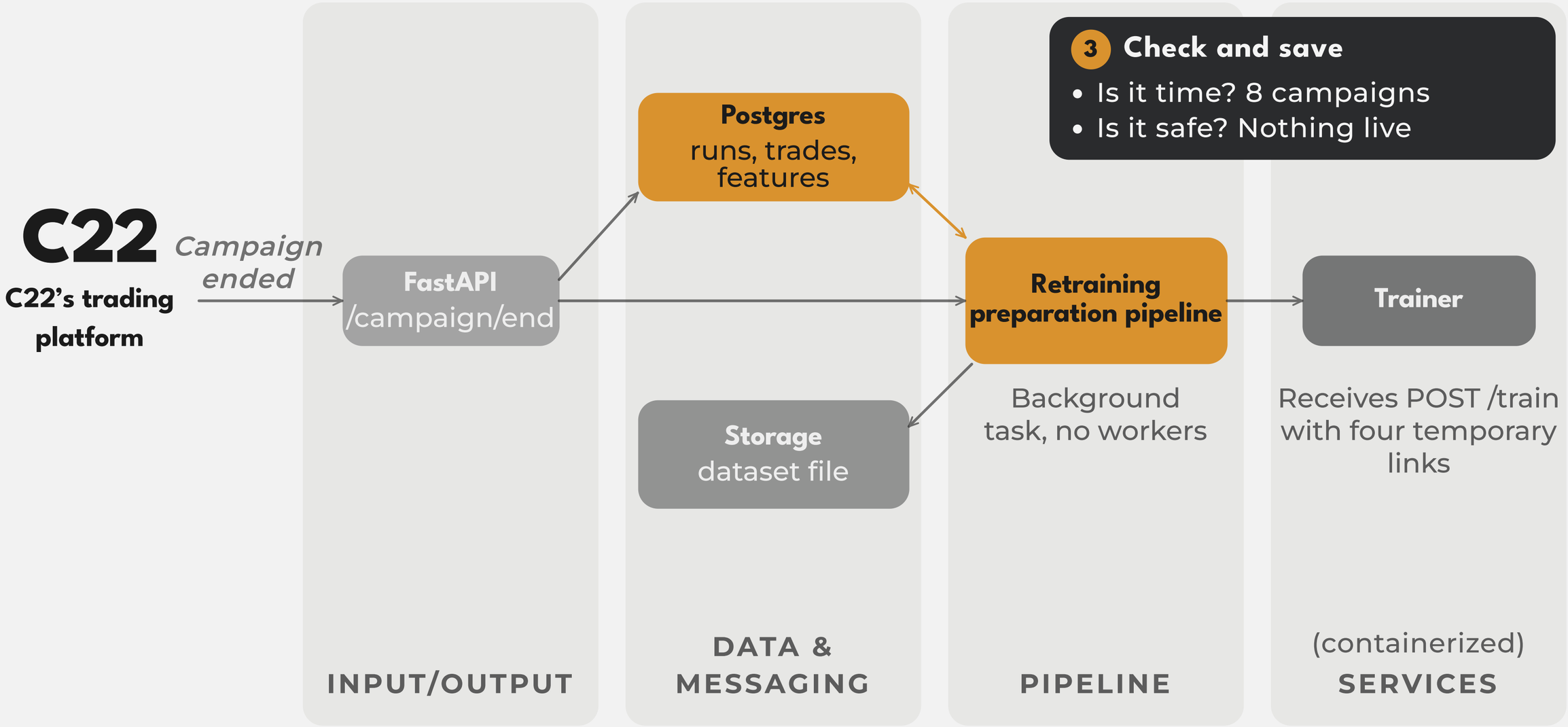
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



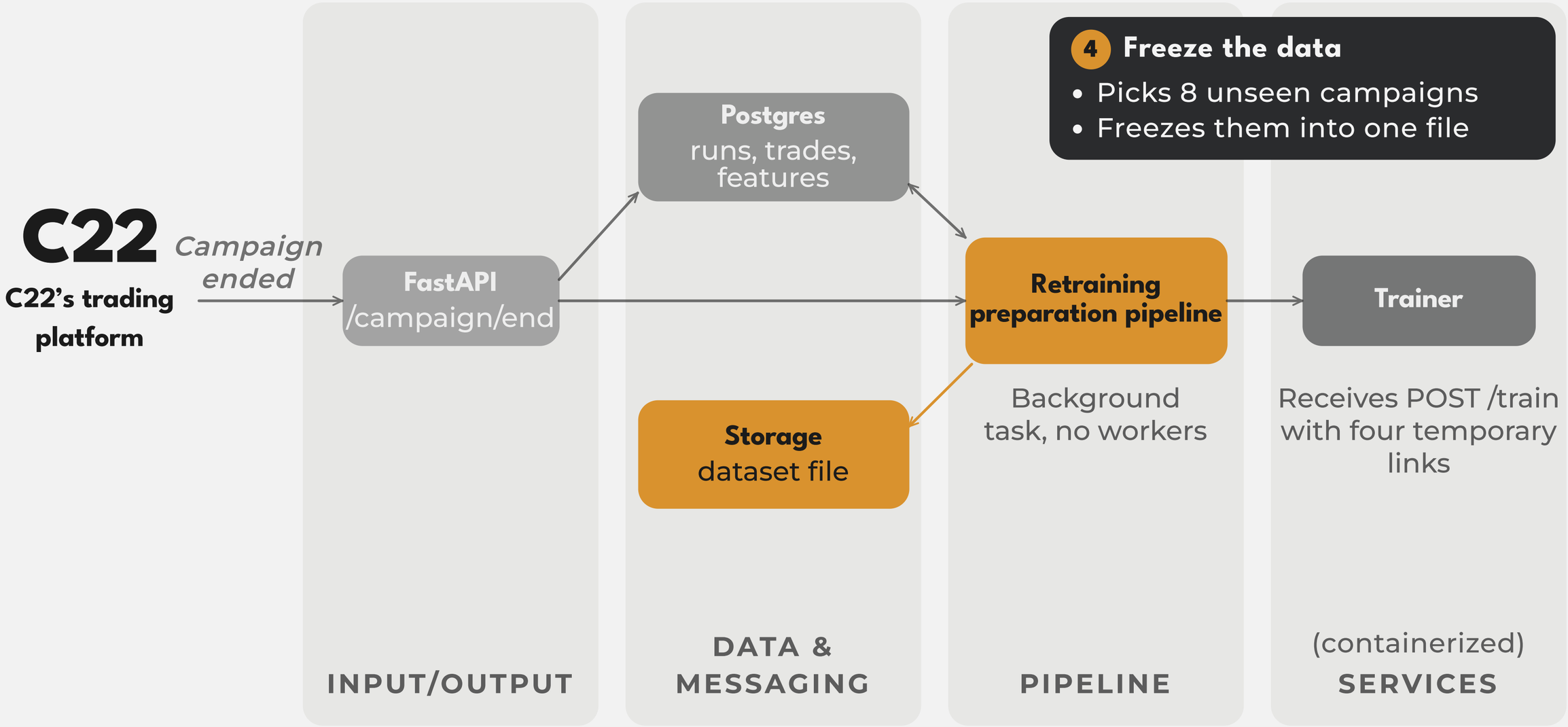
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



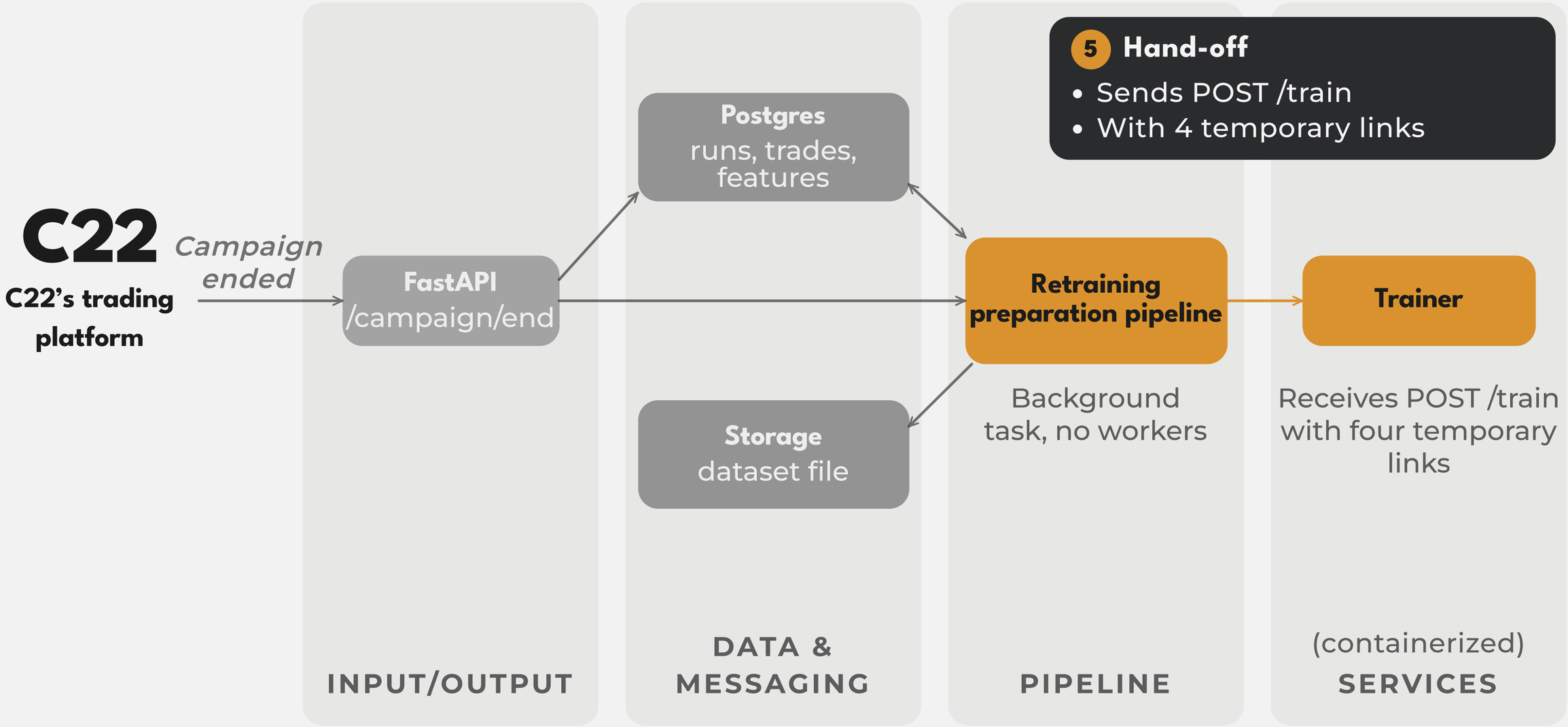
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



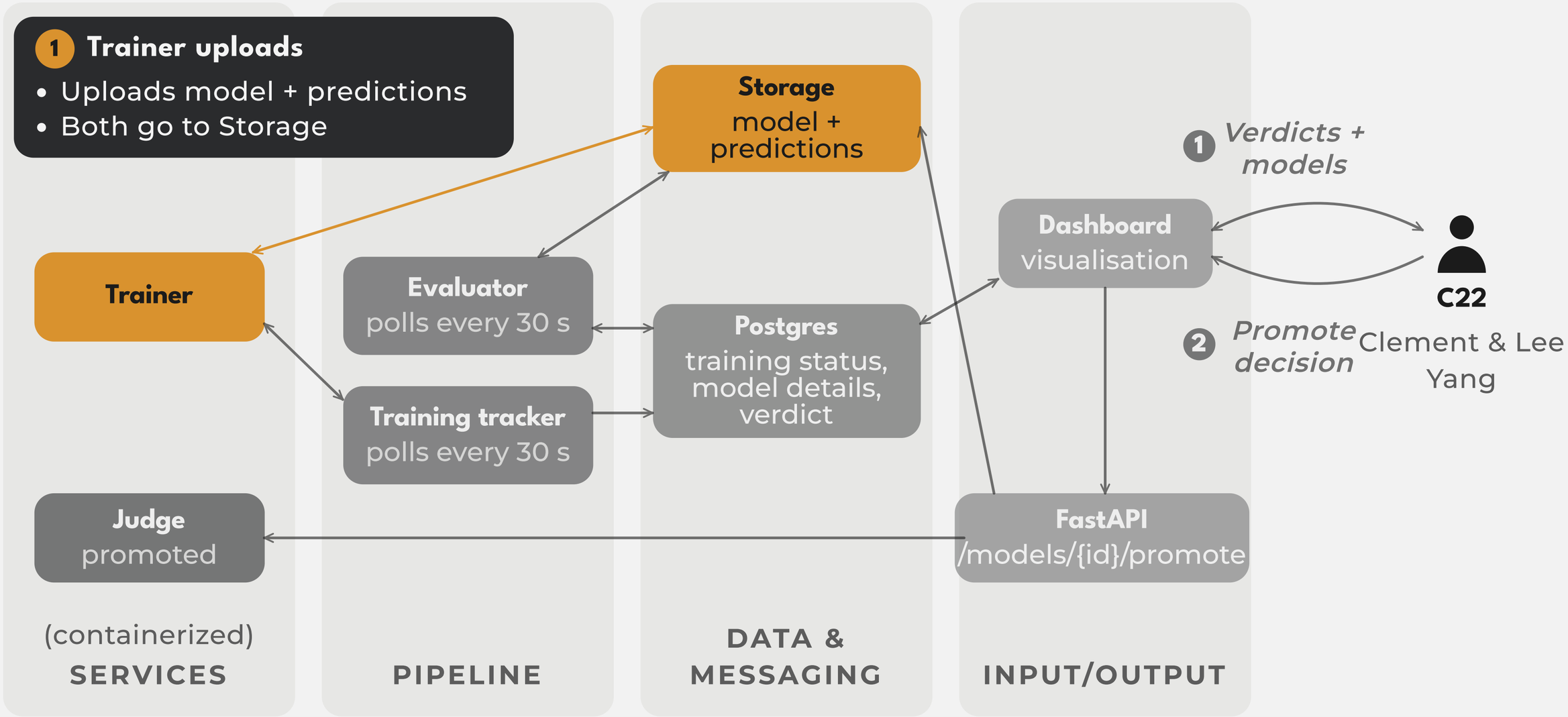
RETRAINER (1/2) - CAMPAIGN END TO PREPARING FOR A RETRAINING SESSION

Project Cassandra Architecture



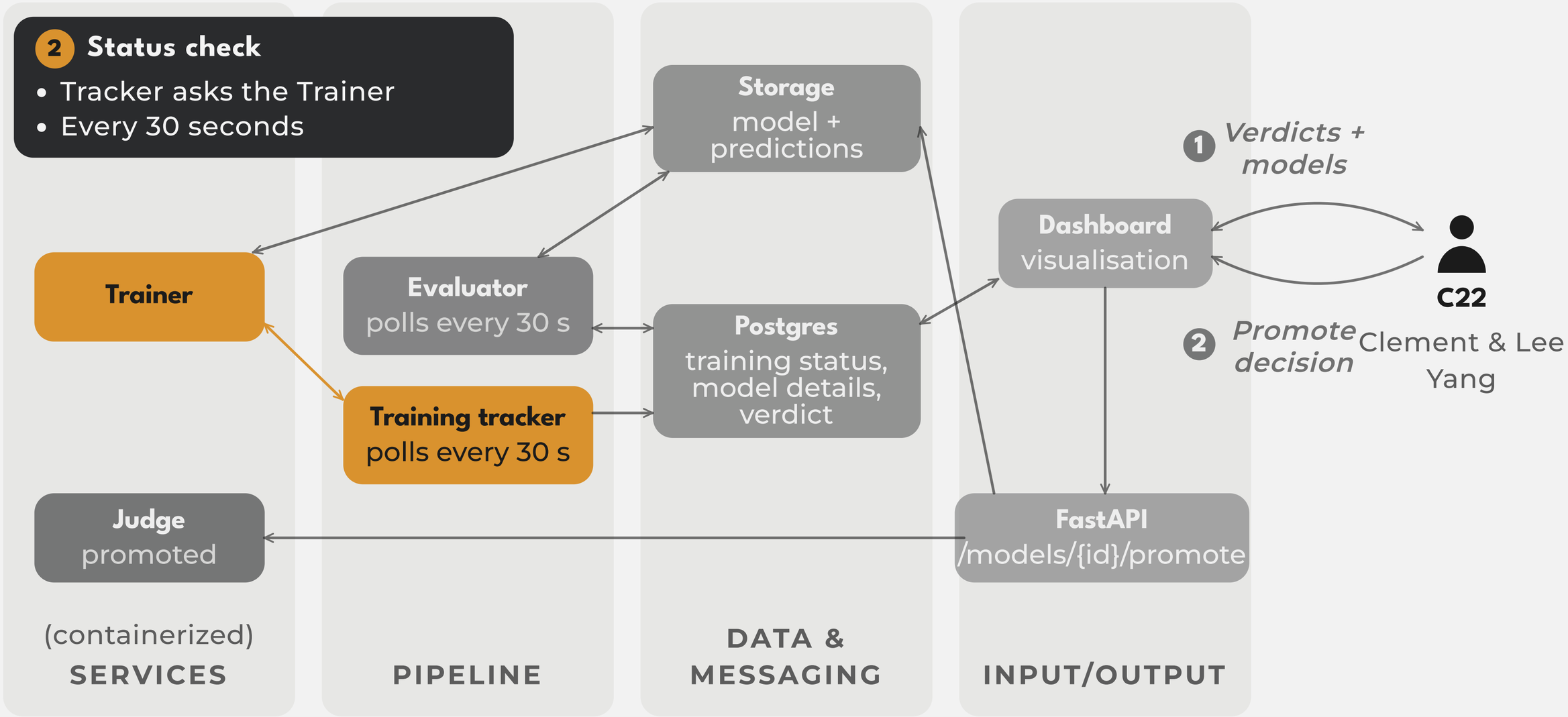
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



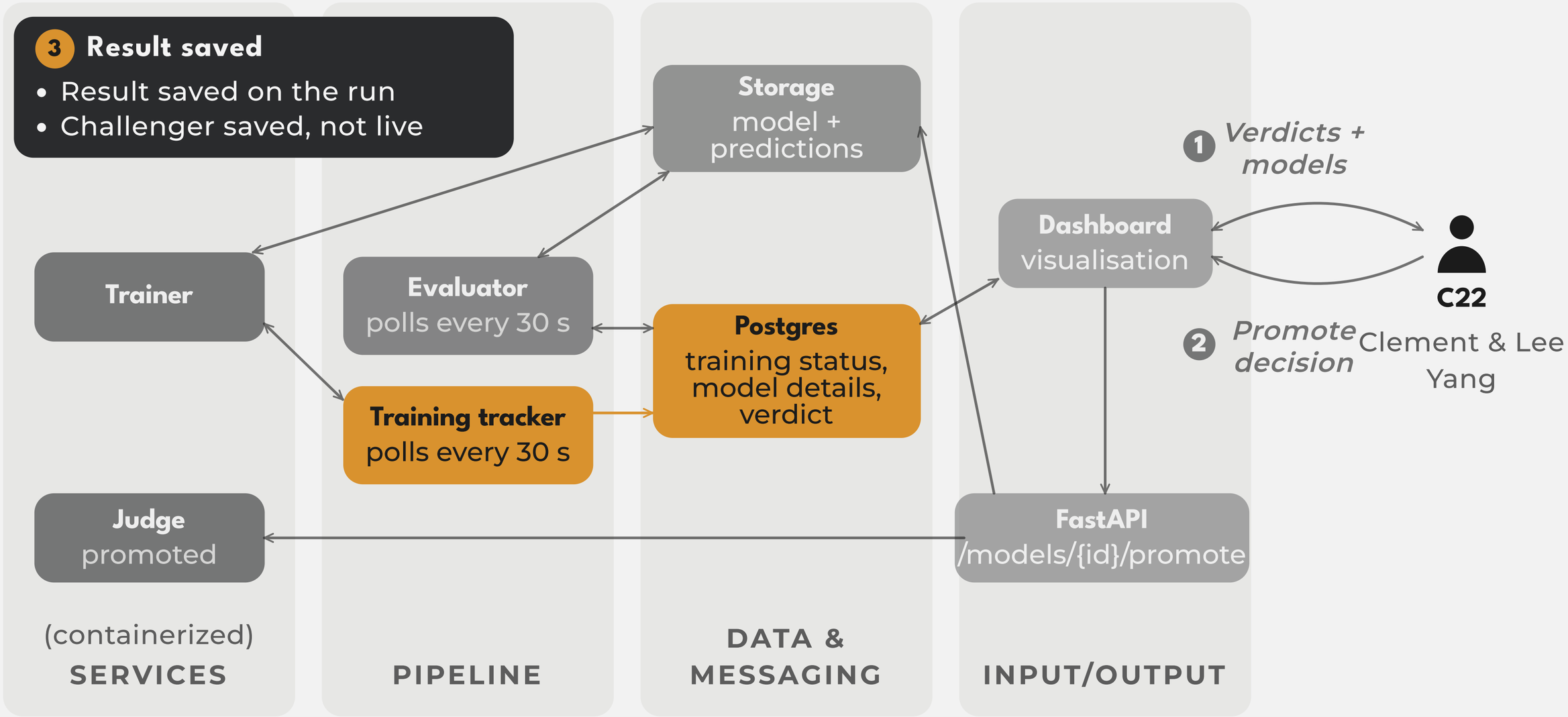
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



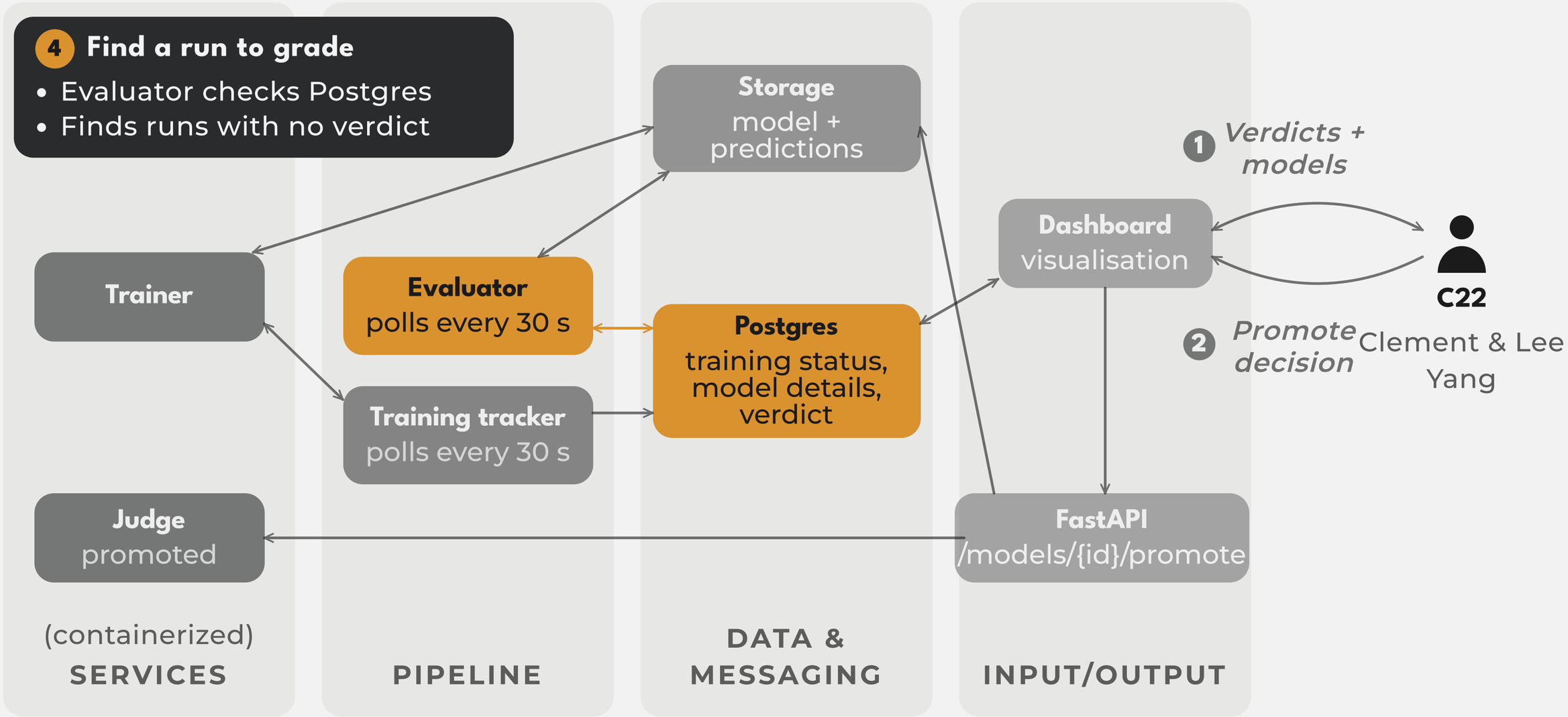
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



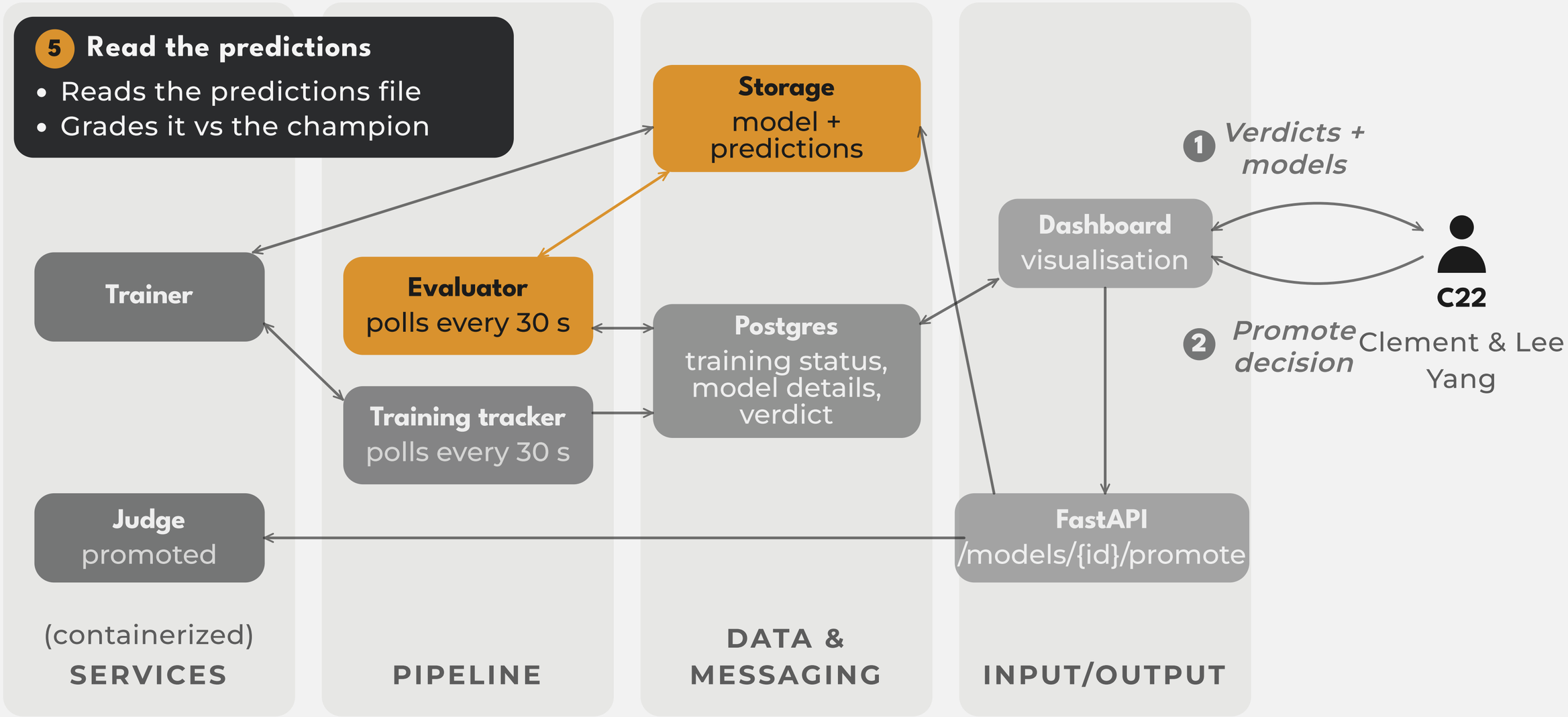
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



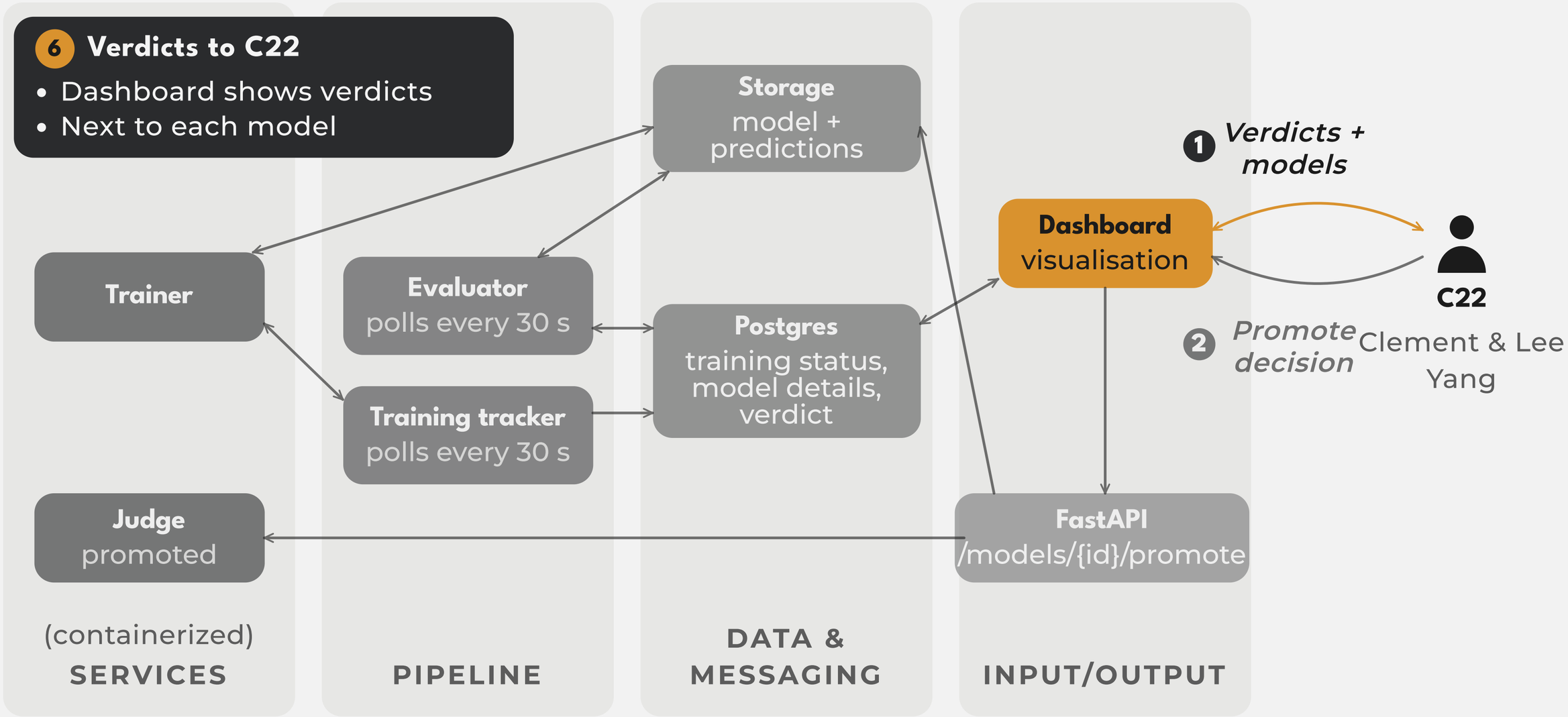
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



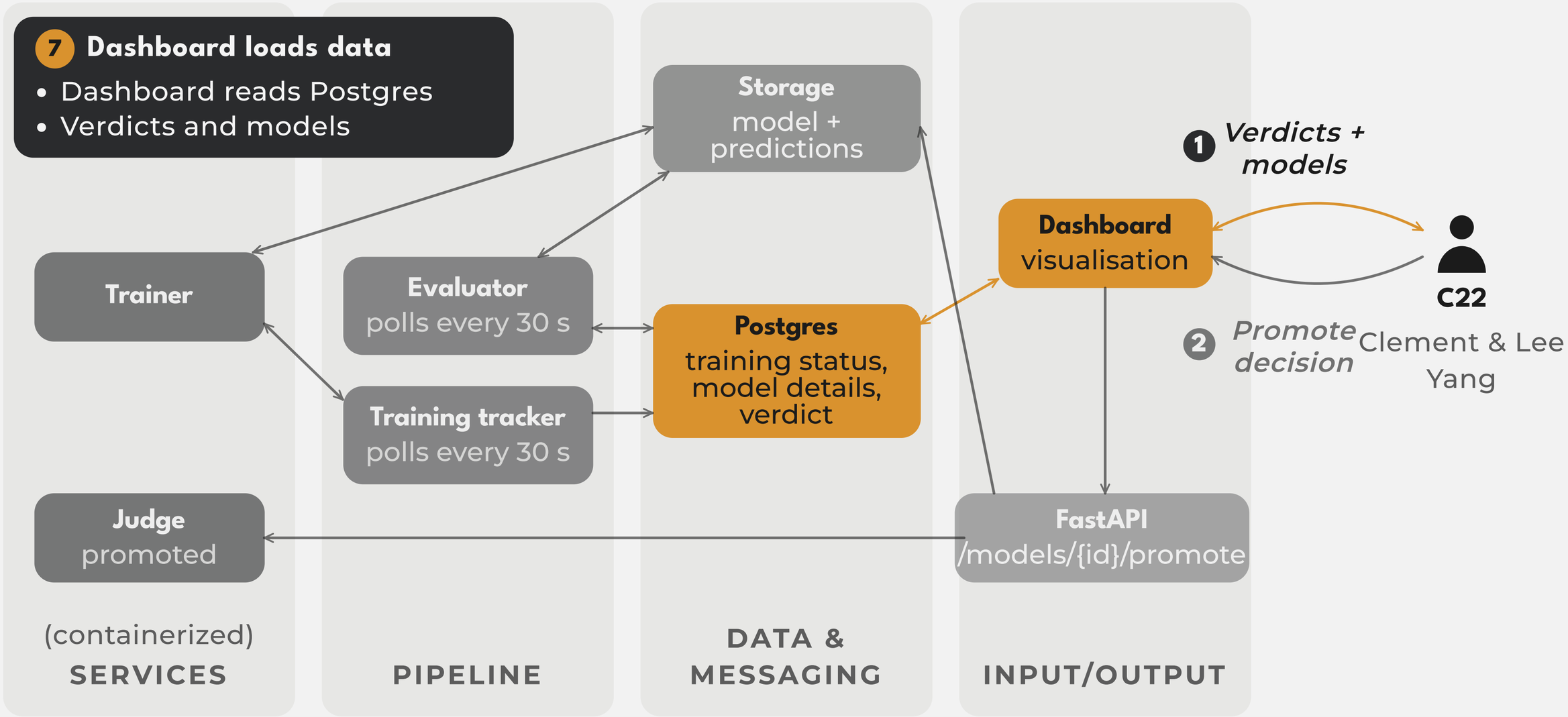
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



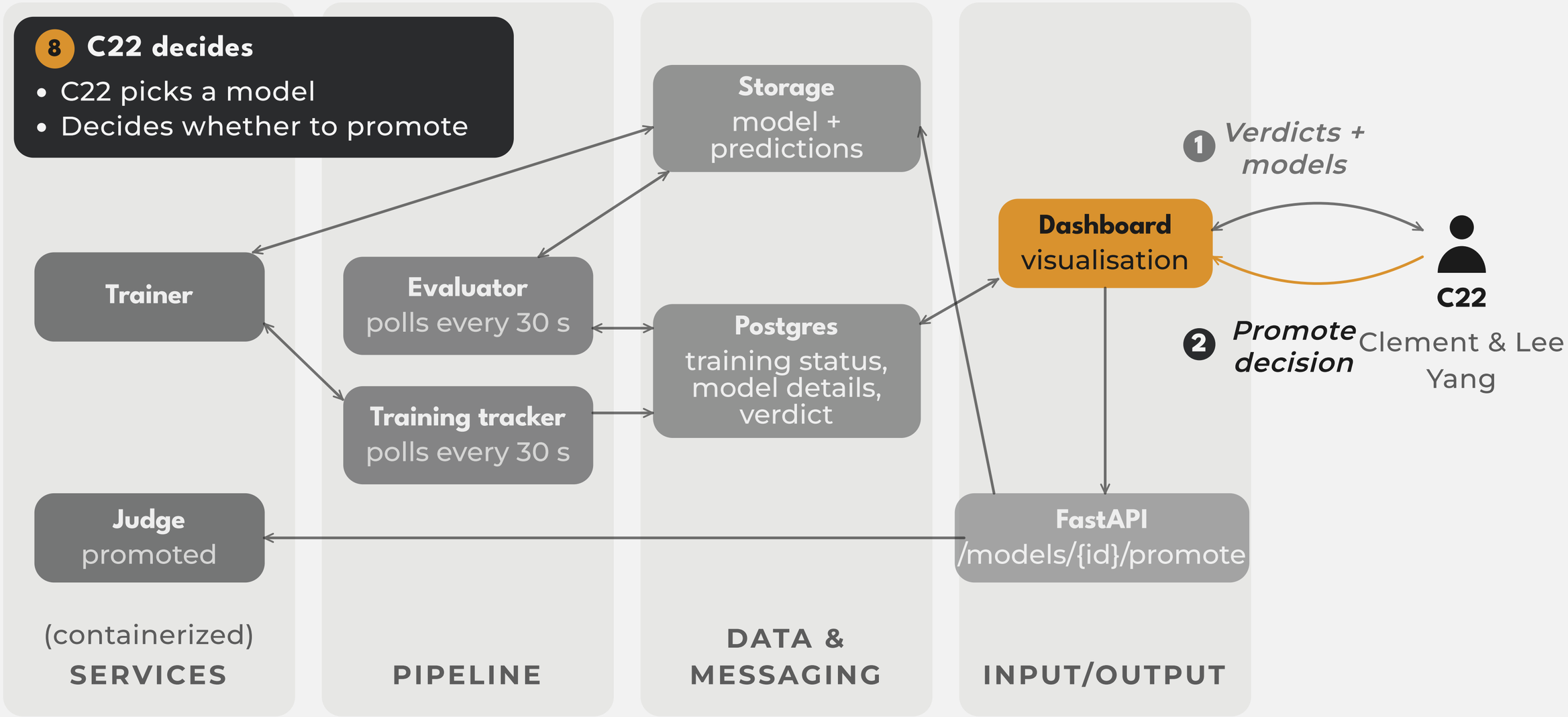
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



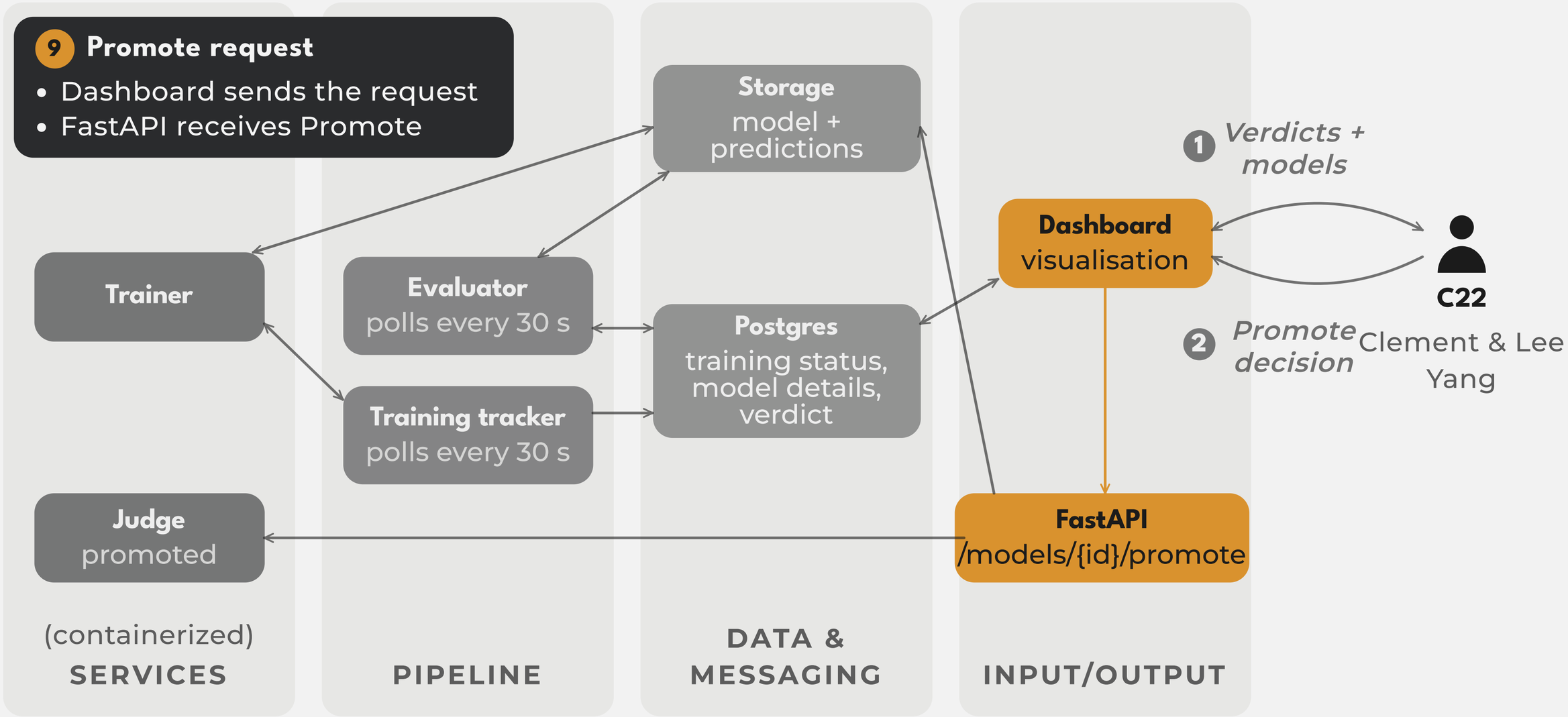
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



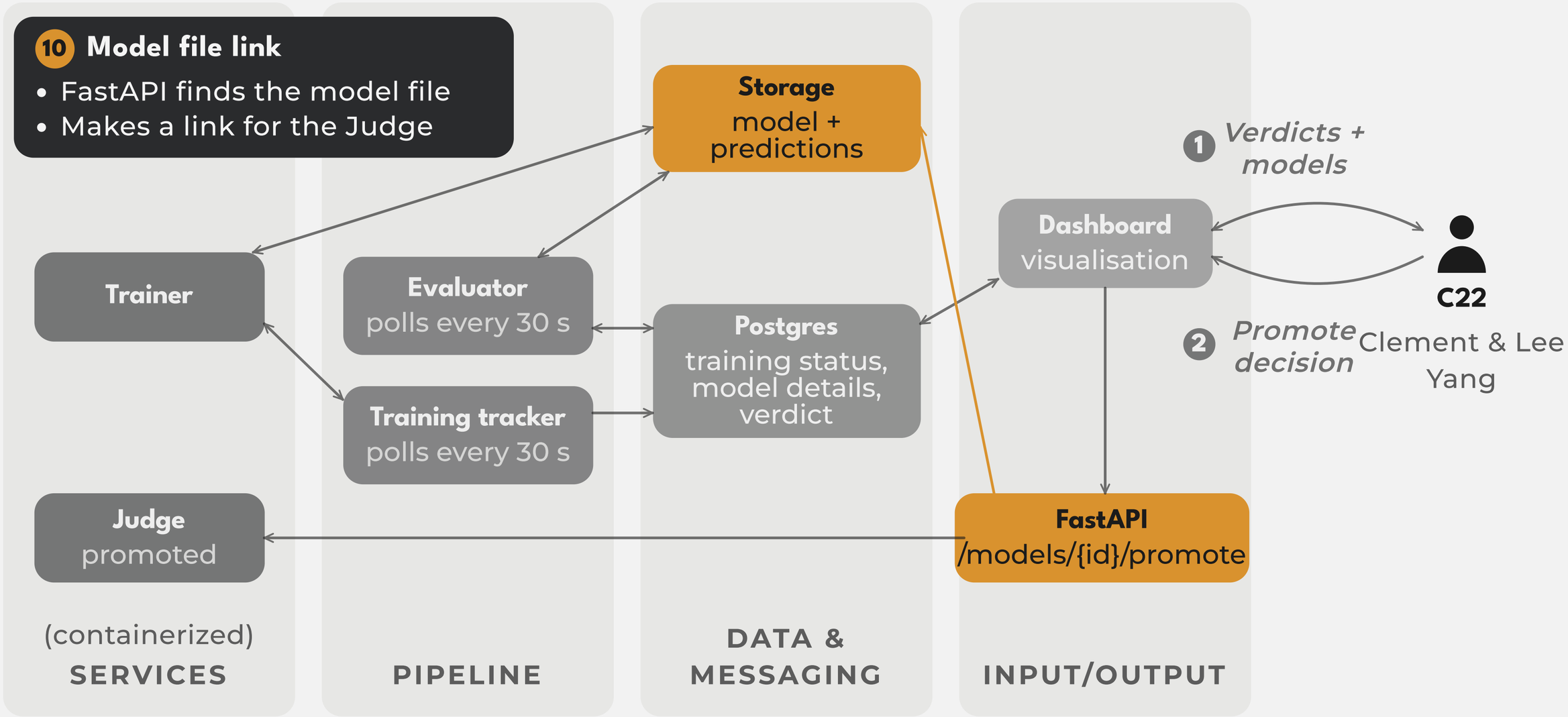
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



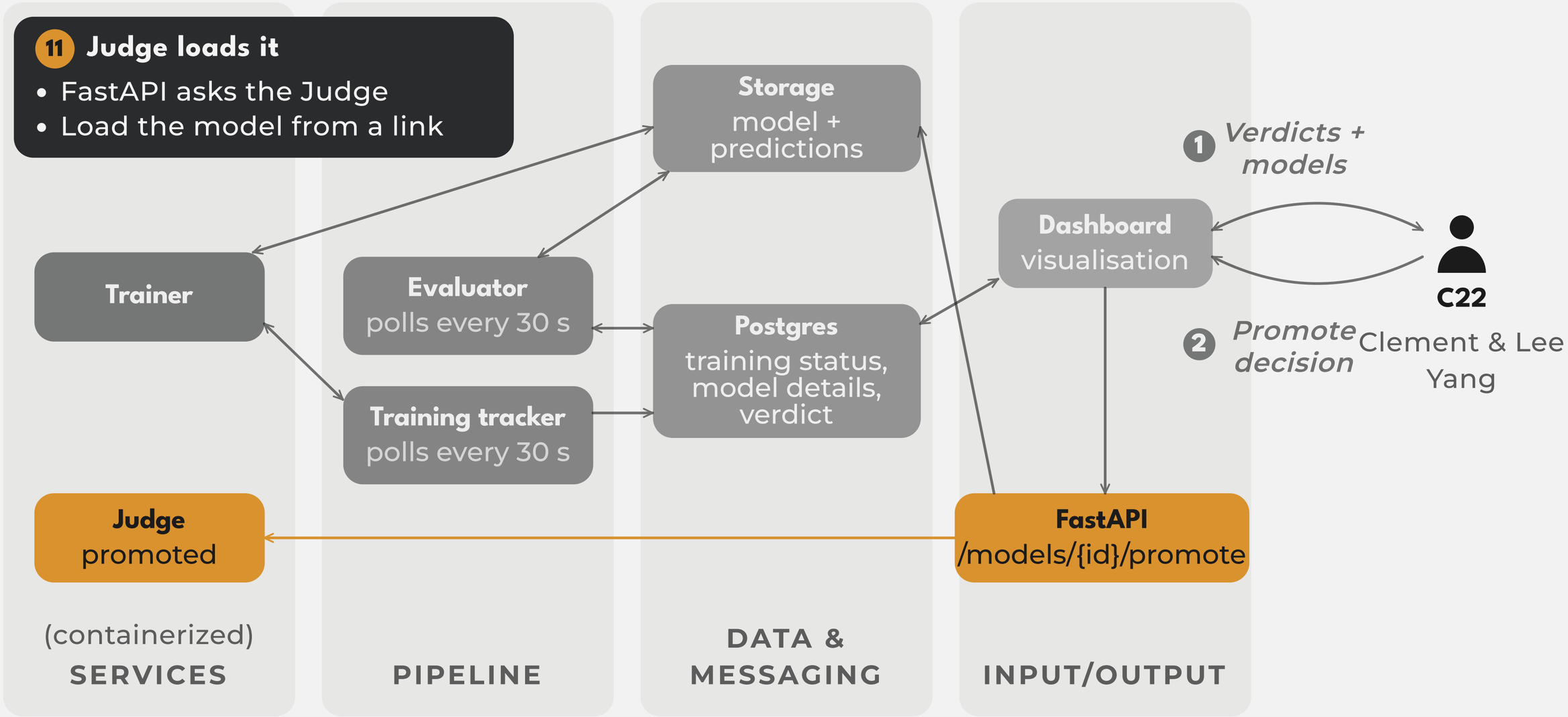
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



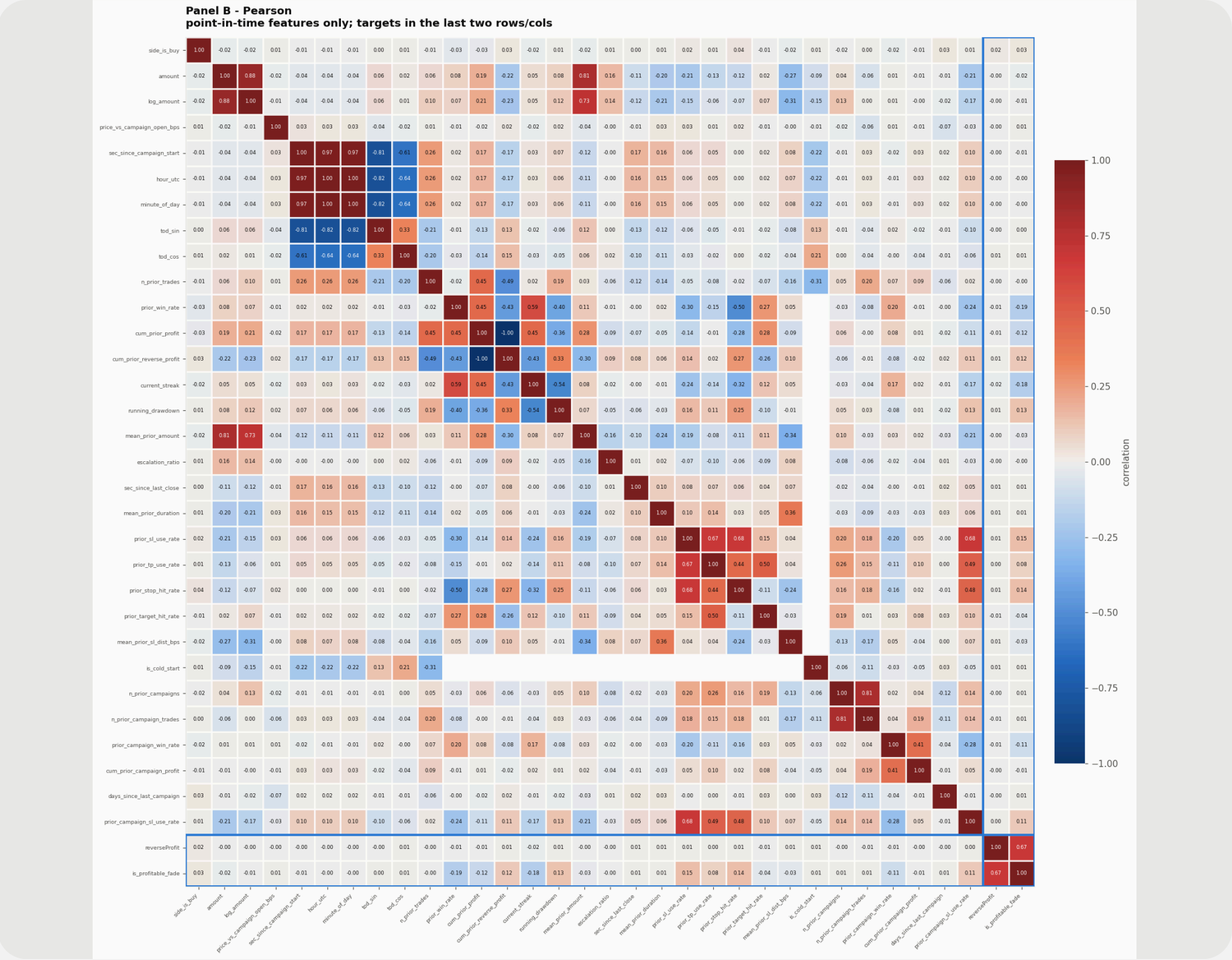
RETRAINER (2/2) - RETRAINING TO PROMOTION OF NEW MODEL TRAINED

Project Cassandra Architecture



APPENDIX

Feature Correlations



Against the target

No feature correlates above 0.19 with whether fading wins

Among themselves

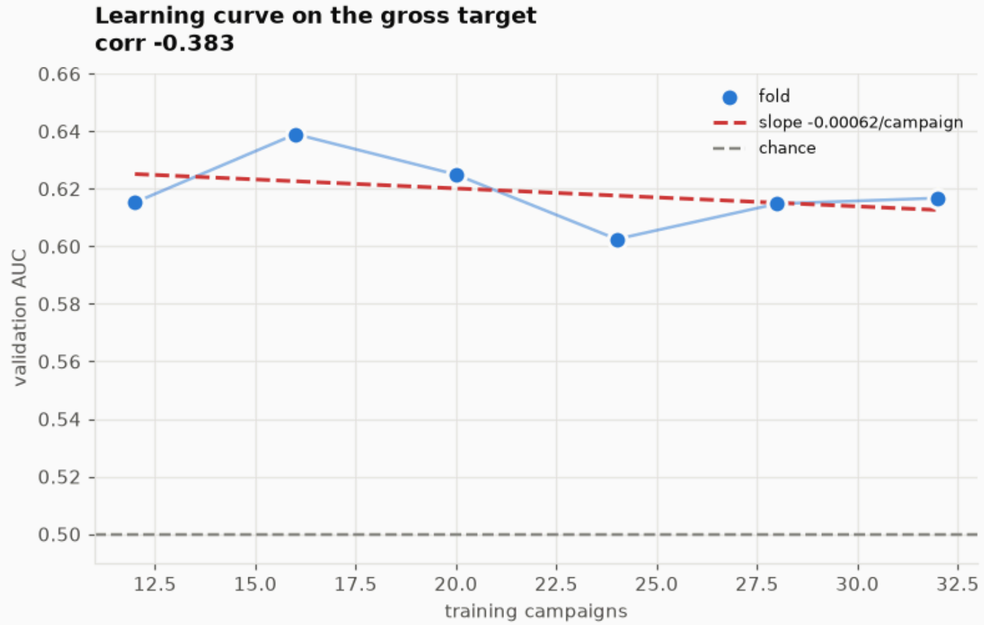
Time-of-day and cumulative-profit features move together, so we read importance with SHAP, not raw correlation

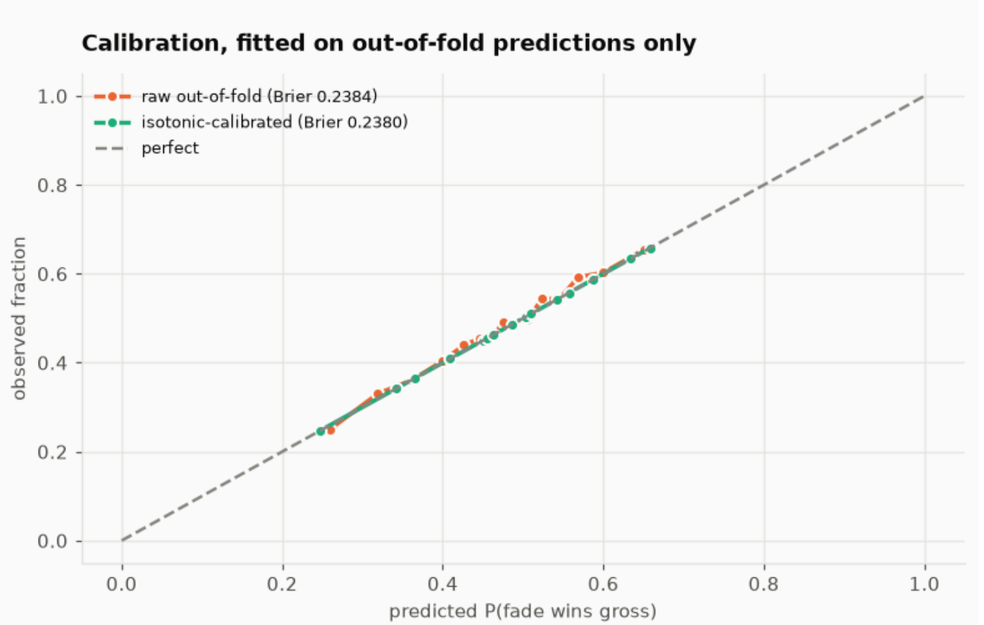
APPENDIX

Learning Curve & Calibration

GROSS TARGET · BLOCKED CV

More campaigns do not help yet

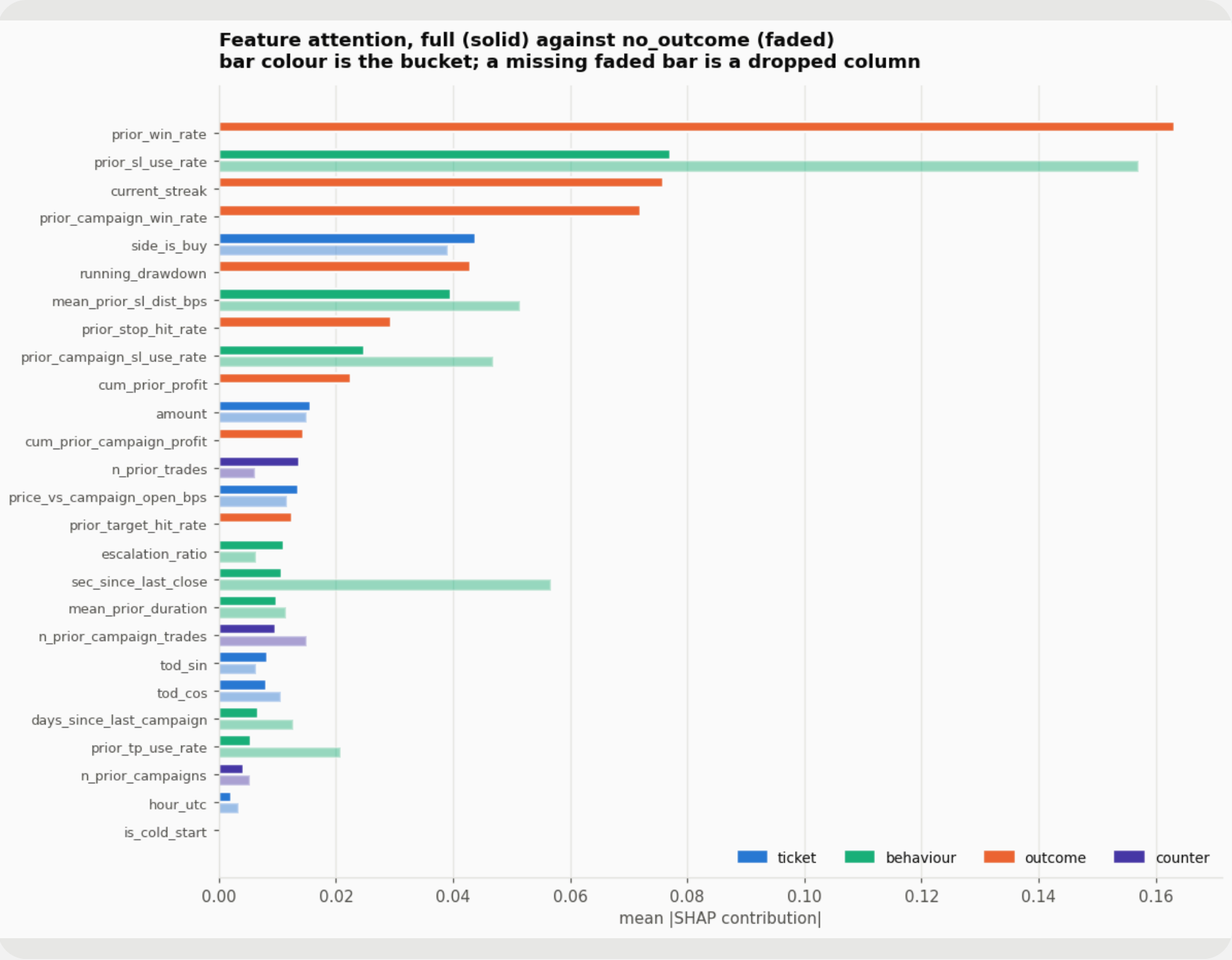




Flat learning curve (corr −0.38)
Brier 0.2384 → 0.2380 after calibration

APPENDIX

SHAP by Feature



Full model

prior_win_rate alone carries 0.16 mean |SHAP|, then prior_sl_use_rate and current_streak

Without outcome history

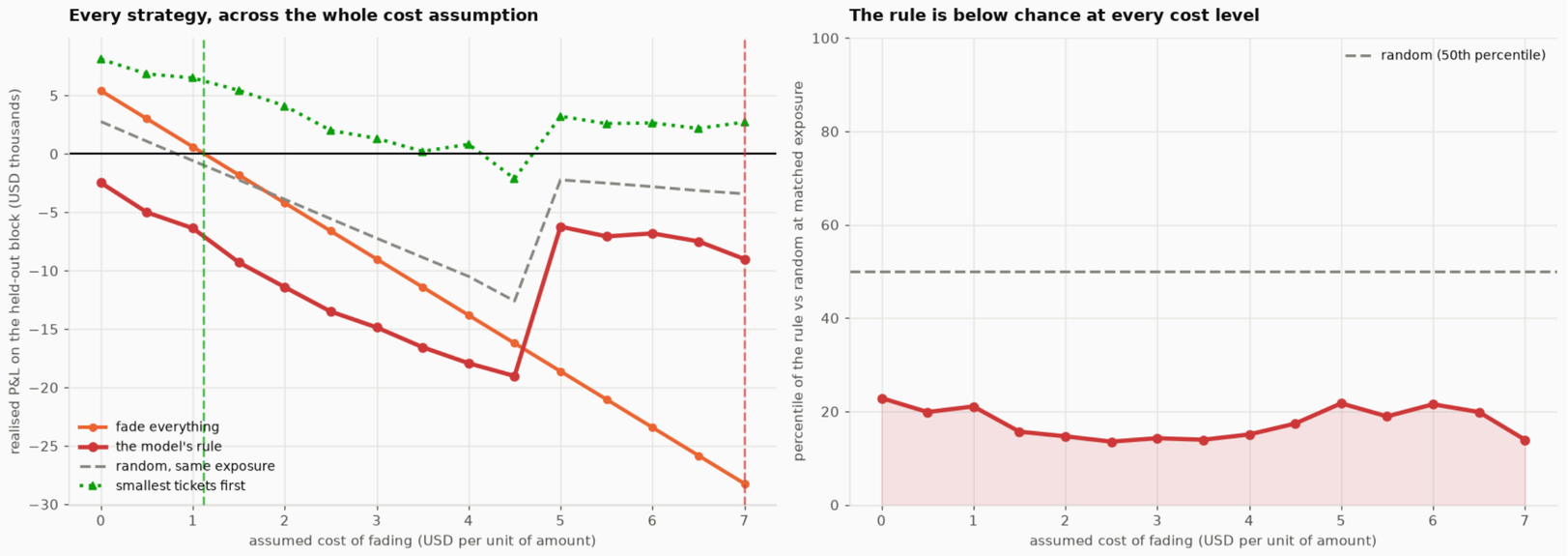
prior_sl_use_rate takes over, so the model falls back on how traders behave

APPENDIX

Cost Assumption Sweep

HELD-OUT CAMPAIGNS 69–77

Every strategy across every cost



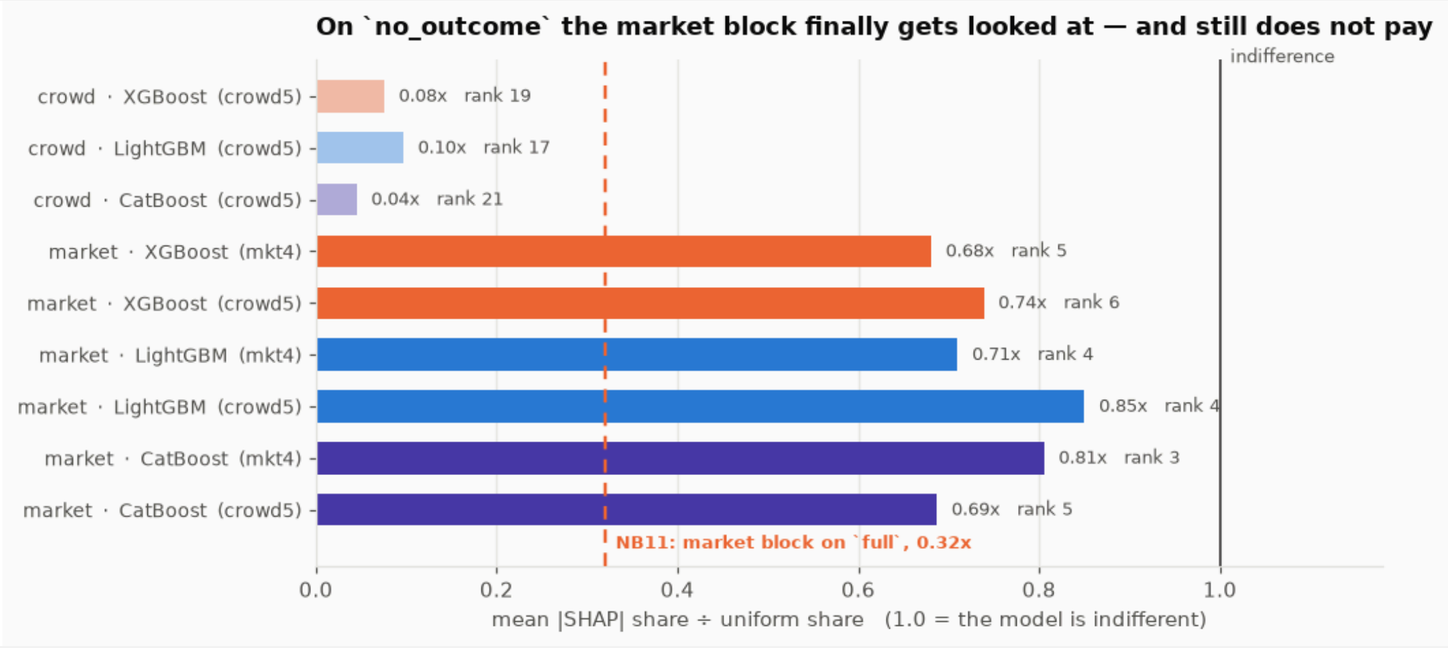
The threshold rule sits at the 13th to 23rd percentile of random at every cost level

APPENDIX

Market Data, Revisited

MEAN |SHAP| SHARE ÷ UNIFORM SHARE

Market features get looked at, and still do not pay



AUC neutral to better
Dollars worse in all eight arms

APPENDIX

Tried and Dropped

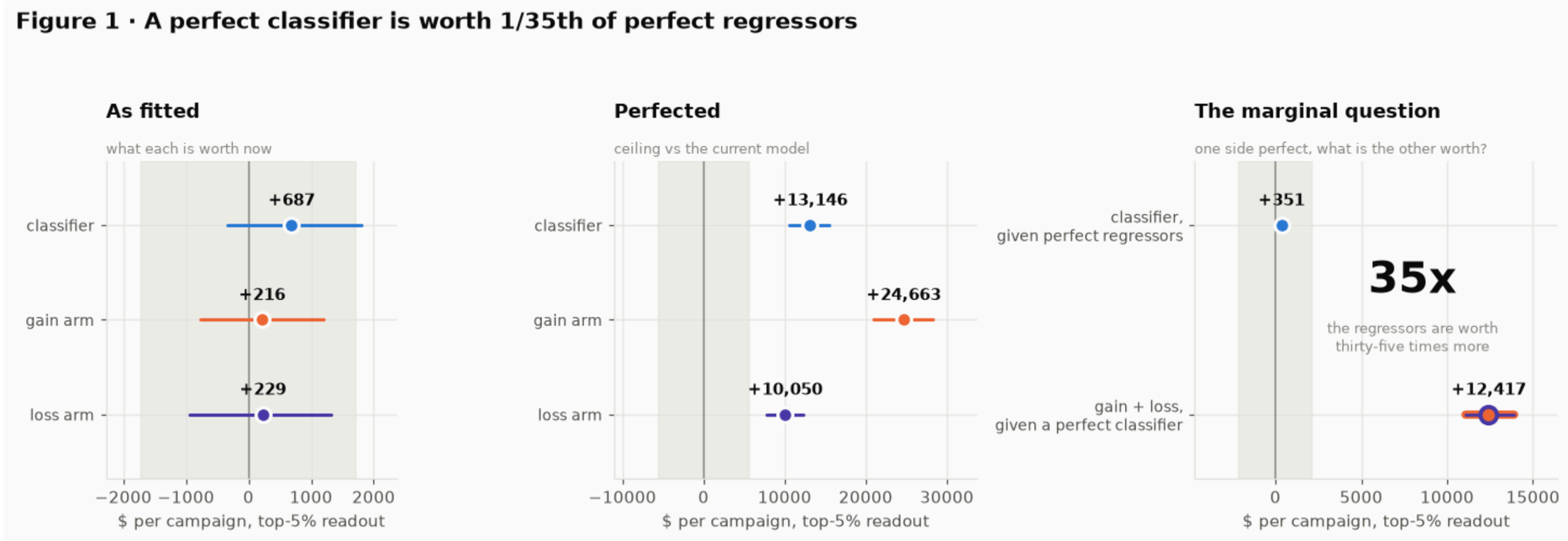
Lever	Result
Market context	Rejected 4×
Stop / target channel	+0.006 AUC at best
Hyperparameter tuning	+0.0028 CV AUC
Tuned GBDT vs logistic	+0.002 AUC
More training campaigns	Flat learning curve
Same-day trade-context block	−$7,571 at top 5%

APPENDIX

Hurdle Decomposition

TOP-5% READOUT · $ PER CAMPAIGN

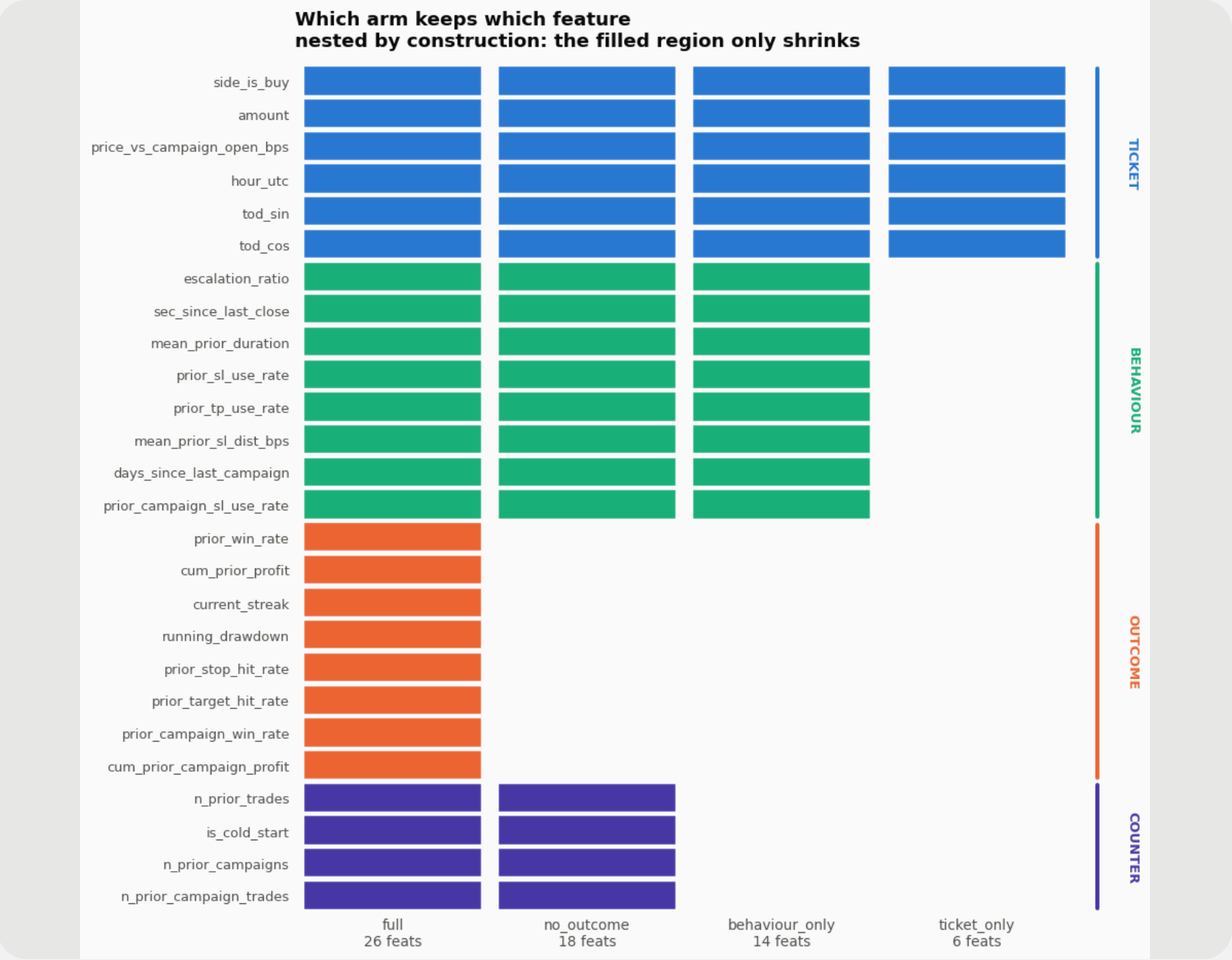
A perfect classifier is worth 1/35th of perfect regressors



+$351 from a perfect classifier
+$12,417 from perfect regressors

APPENDIX

The 26 Features



Four arms, nested

Full (26) → no outcome (18) → behaviour only (14) → ticket only (6)

Outcome goes first

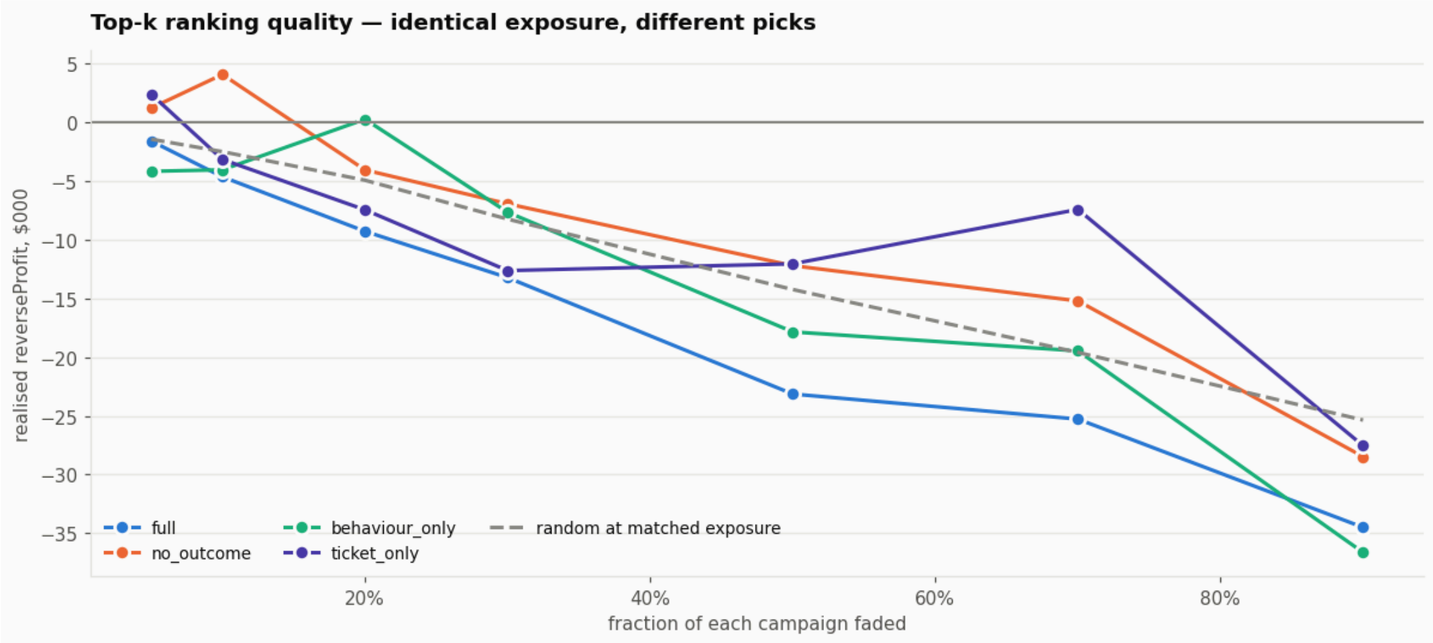
Win/loss history is the block we remove to test behaviour against luck

APPENDIX

Does the Ranking Make Money?

HELD-OUT CAMPAIGNS 69–77

Top-k ranking quality, identical exposure



Full model: below random at every cut
No arm clearly beats random

APPENDIX

Judge v5 Hyperparameters

Tree shape

Shared by all three models

max_depth	4
num_leaves	15
min_child_samples	47
learning_rate	0.0707

Sampling

Shared; includes L1/L2 penalties

subsample	0.752
colsample_bytree	0.638
reg_lambda	4.125
reg_alpha	2.812

Per model

Early stop 100 rounds, refit at median best

	Loss	Trees
Win?	binary	44
Gain	huber	6,392
Loss	huber	6,412

APPENDIX

How We Picked the Demo Data

Two lenses

How busy the slot is, and how volatile the market is



Activity — Trades per slot, p50 vs p95

Volatility — Beta (β) vs Gold Futures

APPENDIX

The Data Behind Our Choice

ACTIVITY

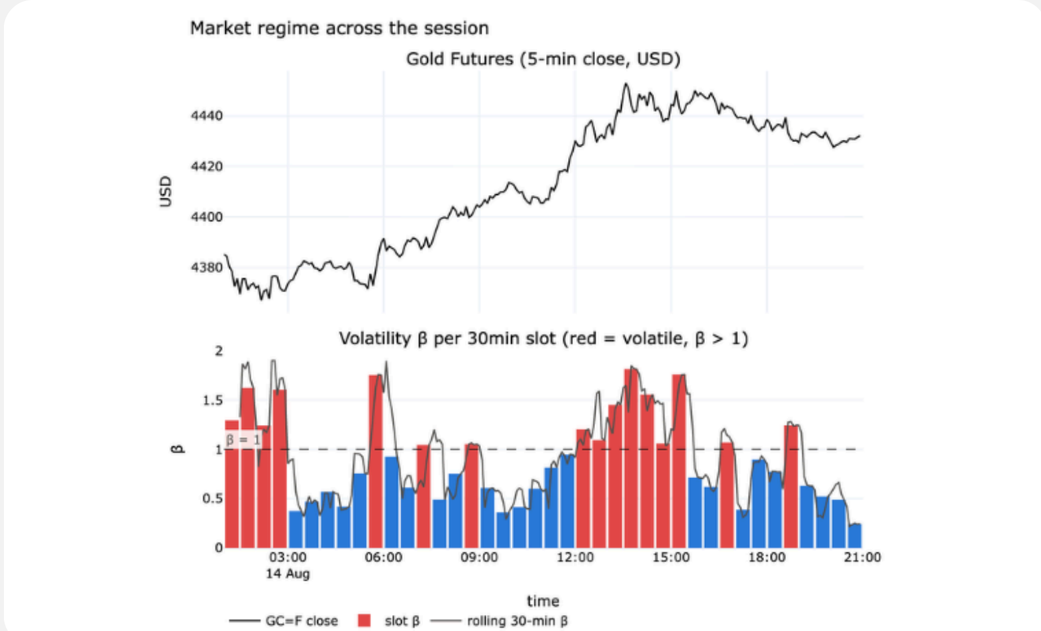
Trades per slot



Typical ≈ p50
Busy = p95 and above

VOLATILITY

Beta vs Gold Futures



Calm: β ≤ 1
Volatile: β > 1

APPENDIX

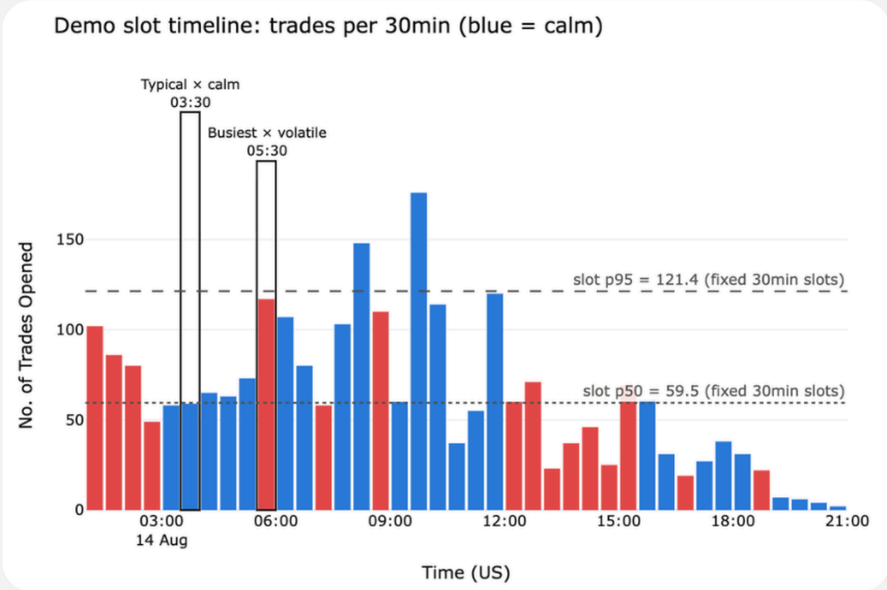
Chosen Timeslots

TRADING ACTIVITY	Calm	Volatile
Typical	DEMO SLOT **Typical · Calm** 3:30 – 4:00	Typical · Volatile
Busy	Busy · Calm	DEMO SLOT **Busy · Volatile** 5:30 – 6:00

PRICE VOLATILITY

FIXED 30-MIN SLOTS · US TIME, 14 AUG

Demo slot timeline



p95 = 121.4 trades
p50 = 59.5 trades